Python Fundamentals

Date : 22/08/25
Page No. : 1

☆ Before NumPy/Pandas, make sure you are comfortable with:

1. Python Basics

☐ Variables and data types

```
a = 10          # int
b = 3.14        # float
c = "hello"     # str
d = True        # bool
```

Type	Example	Description
int	10	Integer numbers
float	3.14	Decimal numbers
str	'hello'	Text / string
bool	True	True or False

☐ Type conversion

```
int("10")     → 10
float("3.5")  → 3.5
str(123)      → '123'
bool(0)       → False
```

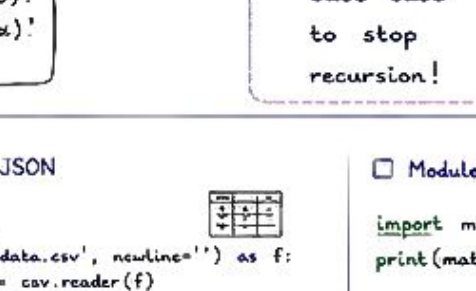
☐ Operators

```
Arithmetic:  +  -  *  /  //  %  **
Comparison:  ==  !=  >  <  >=  <=
Logical:     and  or  not
Assignment:  =  +=  -=  *=  /=
```

2. Control Flow ☆

☐ if / elif / else

```
x = 10
if x > 0:
    print("positive")
elif x == 0:
    print("zero")
else:
    print("negative")
```



☐ for loop

```
for i in range(1, 5):
    print(i)

# Output: 1 2 3 4
```

☐ while loop

```
i = 1
while i <= 4:
    print(i)
    i += 1
```

☐ break, continue, pass

```
for i in range(5):
    if i == 2:
        continue    # skip 2
    if i == 4:
        break       # stop at 4
    print(i)
else:
    pass            # do nothing
```

```
break     → exit loop
continue  → next iteration
pass      → placeholder
```

3. Data Structures ☆

☐ Lists (ordered, mutable)

```
L = [1, 2, 3, 'a', 4.5]
```

1	2	3	'a'	4.5
0	1	2	3	4

☐ Sets (unique, unordered)

```
S = {1, 2, 3, 3} → {1, 2, 3}
```



☐ Tuples (ordered, immutable)

```
T = (1, 2, 'a', 4.5)
```

☐ Dictionaries (key-value pairs)

```
D = {'name': 'Alice', 'age': 20}
```

key	value
'name'	'Alice'
'age'	20

☐ Nested data structures

```
nested = {'nums': [1, 2, 3], 'info': {'age': 20}}
```

☐ Slicing

```
L = [10, 20, 30, 40, 50]
L[1:4]  → [20, 30, 40]
L[::3]  → [10, 20, 30]
L[::2]  → [10, 30, 50]
```

0	1	2	3	4
10	20	30	40	50

☐ Comprehensions

```
List :  [x*x for x in range(5)]
Set  :  {x for x in range(5)}
Dict :  {x: x*x for x in range(5)}

# Output → [0, 1, 4, 9, 16]
```

4. Functions ☆

☐ Defining functions

```
def add(a, b):
    return a + b
add(2, 3)    # 5
```

☐ Parameters and arguments

```
def greet(name, msg):
    print(f"{msg}, {name}")
greet("Alice", "Hello")
```

☐ *args, **kwargs

```
def demo(*args):
    print(args)      # tuple
def info(**kwargs):
    print(kwargs)    # dict
```

☐ Lambda functions

```
square = lambda x: x*x
square(4)    # 16
```

☐ map(), filter(), zip()

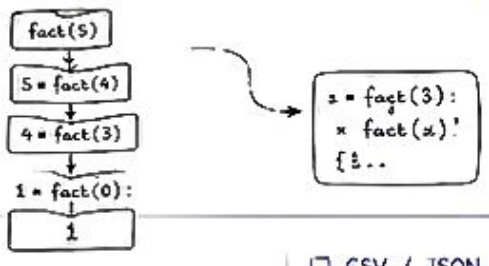
```
nums = [1, 2, 3, 4]
squares = list(map(lambda x: x*x, nums))
evens = list(filter(lambda x: x%2==0, nums))
pairs = list(zip(nums, squares))
```

☐ Scope

```
x = 10,      # global
def f():
    x = 5    # local
    print(x)
f()   # 5
print(x)  # 10
```

☐ Recursion basics

```
def fact(n):
    if n == 0:
        return 1
    return n * fact(n-1)
fact(5)    # 120
```



- Always define base case to stop recursion! ☆

5. Python for Data Work ☆

☐ Exception handling

```
try:
    x = 10/0
except ZeroDivisionError:
    print("Cannot divide by 0")
else:
    print("No error")
finally:
    print("Done")
```

☐ File handling

```
# Read
with open('file.txt','r') as f:
    data = f.read()

# Write
with open('out.txt','w') as f:
    f.write("Hello")
```

☐ CSV / JSON

```
# CSV
import csv
with open('data.csv', newline='') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)

# JSON
import json
data = json.load(open('data.json'))
```

☐ Modules and packages

```
import math
print(math.sqrt(16))

from collections import Counter

from mypkg import utils
```

☐ datetime

```
from datetime import datetime
now = datetime.now()
print(now)
print(now.strftime("%Y-%m-%d"))
```

☐ Iterators and generators

```
# Iterator
it = iter([1,2,3])
next(it)  # 1
next(it)  # 2
```

```
# Generator
def count(n):
    for i in range(n):
        yield i
for x in count(3):
    print(x)
```

☐ Basic OOP

```
class Person:
  def __init__(self, name, age):
      self.name = name
      self.age = age
  def greet(self):
      return f"Hi, I am {self.name}"
p = Person("Alice", 20)
print(p.greet(1)
```

☐ Virtual environments

```
# Create
python -m venv myenv

# Activate
# Windows
myenv\Scripts\activate
# macOS / Linux
source myenv/bin/activate
```

☐ pip

```
# Install package
pip install numpy

# List installed packages
pip list

# Freeze requirements
pip freeze > requirements.txt
```

Tip: Practice daily. Read docs. Write code. Debug. Repeat!

Happy Coding!

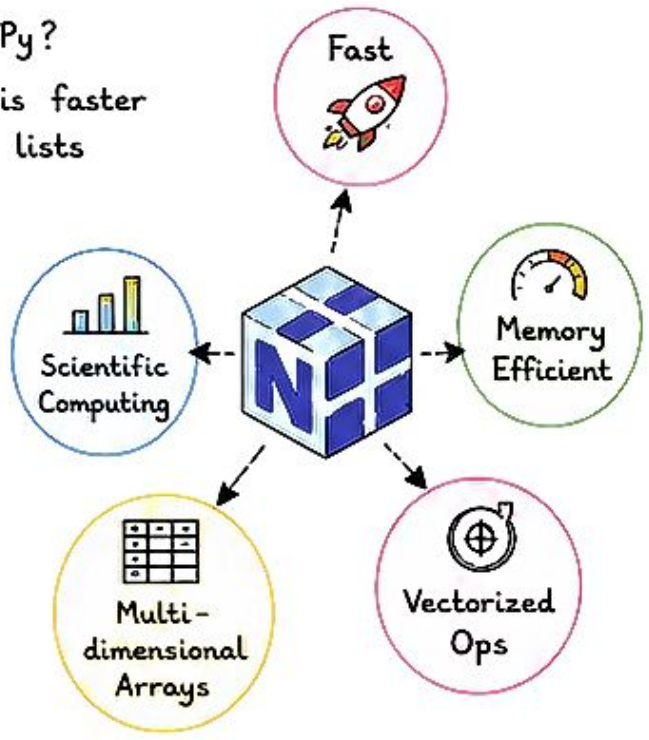
NumPy – Beginner

Date : 22/08/25
Page No. : 1

NumPy is the foundation for scientific Python and much of the numerical machinery behind Pandas.

1.1 NumPy Fundamentals

- [ ] What is NumPy?
- [ ] Why NumPy is faster than Python lists
- [ ] ndarray
- [ ] Dimensions
- [ ] Axes
- [ ] Shape
- [ ] Size
- [ ] dtype
- [ ] ndim
- [ ] itemsize
- [ ] nbytes

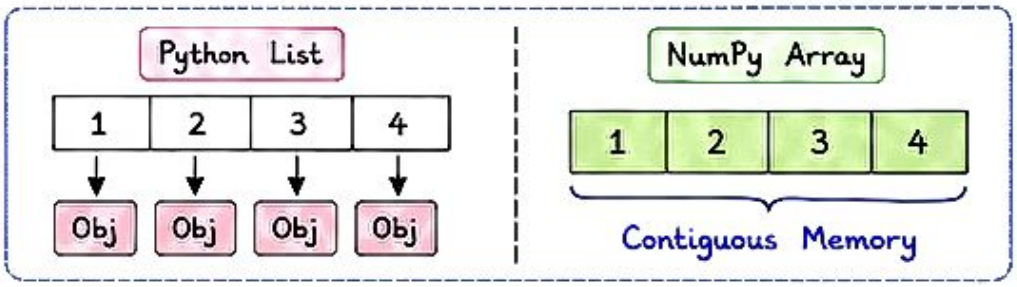


What is NumPy?

- NumPy (Numerical Python) is a library for working with arrays and matrices.
- It provides high-performance multi-dimensional array objects and tools to work with them.

Why NumPy is faster than Python lists?

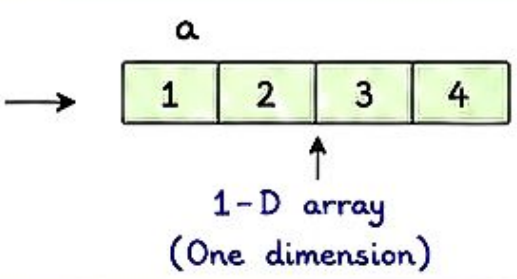
- ✓ Implemented in C (optimized).
- ✓ Stores data in contiguous memory block.
- ✓ Avoids Python object overhead.
- ✓ Vectorized operations (apply operation on whole array at once).



Learn:

```
import numpy as np

a = np.array([1, 2, 3, 4])
# 1-D array
```



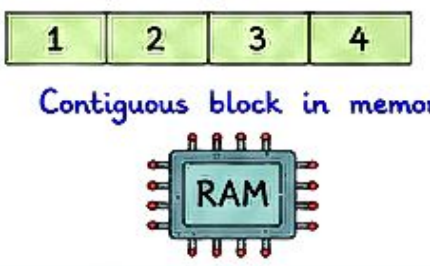
Key idea:
NumPy works with arrays, not lists!

Understand these attributes

Attribute	What it means	Example (a = np.array([1, 2, 3, 4]))	Output / Type
a.shape	Shape of the array (rows, columns, ...)	a.shape	(4,) → tuple
a.ndim	Number of dimensions (ndim = "number of axes")	a.ndim	1 → int
a.size	Total number of elements in the array	a.size	4 → int
a.dtype	Data type of elements in the array	a.dtype	dtype('int64')
a.itemsize	Size (in bytes) of each element	a.itemsize	8 → bytes (on most 64-bit systems)
a.nbytes	Total bytes consumed by the array	a.nbytes	32 → bytes (4 elements × 8 bytes)

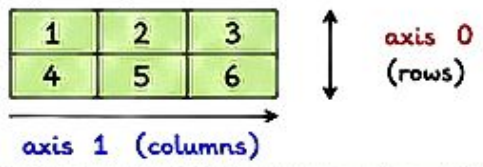
Memory View

```
a = np.array([1, 2, 3, 4])
```



More Dimensions

```
2-D array (matrix)
b = np.array([[1, 2, 3], [4, 5, 6]])
```



Quick Tips

- ✓ Use NumPy for numbers, arrays, matrices, and scientific computing.
- ✓ Prefer NumPy arrays over Python lists for performance.

Remember

Array → Fixed type
List → Flexible type

Creating NumPy Arrays

Master all major array creation techniques.

Date : 22/08/25
Page No. : 1

1. Basic Array Creation (deterministic)

① np.array(object)

Create an array from Python objects (list, tuple, etc.).

Example:

```
np.array([1, 2, 3])  →  [1 2 3]
```

② np.asarray(object)

Convert input to an array (if not already).

Example: `np.asarray((1, 2, 3))` → `[1 2 3]`

③ np.zeros(shape, dtype)

Create an array filled with zeros.

Example: `np.zeros((2, 3), dtype=int)` →

```
0 0 0
0 0 0
```

0

④ np.ones(shape, dtype)

Create an array filled with ones.

Example: `np.ones((2, 3))` →

```
1 1 1
1 1 1
```

1

⑤ np.empty(shape, dtype)

Create an uninitialized array (faster than zeros).

Example: `np.empty((2, 3))` →

```
? ? ?
? ? ?
```

?

⑥ np.full(shape, fill_value, dtype)

Create an array filled with a specific value.

Example: `np.full((2, 3), 7)` →

```
7 7 7
7 7 7
```

7

2. More Creation Functions (deterministic)

⑦ np.eye(N, M=None, k=0)

Create a 2-D array with ones on the kth diagonal.

Example: `np.eye(3)` →

```
[1 0 0]
[0 1 0]
[0 0 1]
```



Identity matrix

⑧ np.identity(n)

Return the identity matrix of size n.

Example: `np.identity(3)` →

```
[1 0 0]
[0 1 0]
[0 0 1]
```

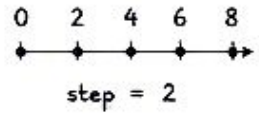
= `np.eye(3)`

⑨ np.arange(start, stop, step)

Return evenly spaced values within a given interval.

Example: `np.arange(0, 10, 2)`

→ `[0 2 4 6 8]`

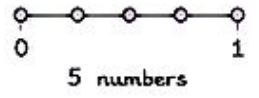


⑩ np.linspace(start, stop, num)

Return num evenly spaced values over [start, stop].

Example: `np.linspace(0, 1, 5)` →

`[0. 0.25 0.5 0.75 1.]`

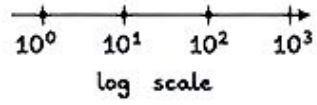


⑪ np.logspace(start, stop, num, base=10)

Return numbers spaced evenly on a log scale.

Example: `np.logspace(0, 3, 4)`

→ `[1. 10. 100. 1000.]`

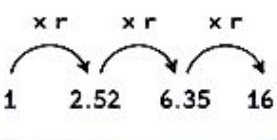


⑫ np.geomspace(start, stop, num)

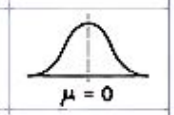
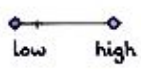
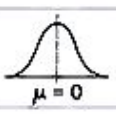
Return numbers spaced evenly on a geometric progression.

Example: `np.geomspace(1, 16, 4)` →

→ `[1. 2.5198421 6.3496042 16.]`



3. Random Arrays (stochastic)

Function	What it does	Example	Output (shape/details)	Visual
① np.random.random(size)	Uniform in [0, 1).	np.random.random(3)	[0.19 0.72 0.33]	
② np.random.rand(d0, d1, ...)	Uniform in [0, 1).	np.random.rand(2, 3)	[[0.11 0.66 0.78] [0.31 0.22 0.93]]	
③ np.random.randn(d0, d1, ...)	Standard normal (mean 0, std 1).	np.random.randn(3)	[0.15 -1.20 0.85]	
④ np.random.randint(low, high, size)	Random integers from [low, high).	np.random.randint(1, 10, 5)	[7 2 9 1 4]	
⑤ np.random.uniform(low, high, size)	Uniform in [low, high).	np.random.uniform(-1, 1, 4)	[-0.43 0.81 -0.10 0.56]	
⑥ np.random.normal(loc, scale, size)	Normal distribution N(loc, scale²).	np.random.normal(0, 1, 4)	[0.32 -0.15 1.21 -0.73]	
⑦ np.random.choice(a, size, replace=True, p=None)	Random sample from 1-D array a.	np.random.choice([1,2,3,4], 3)	[3 1 4]	
⑧ np.random.shuffle(x) Shuffle array in-place.	x = [1, 2, 3, 4] np.random.shuffle(x) → x			

4. Random Seeds / Generators

Modern random API (recommended)

```
rng = np.random.default_rng(42)
```

← Create a Generator with seed 42

Use methods from the generator object:

```
rng.random(size)                    # uniform [0, 1)
rng.integers(low, high, size)       # integers in [low, high)
rng.normal(loc, scale, size)        # normal distribution
rng.uniform(low, high, size)        # uniform [low, high)
rng.choice(a, size, replace=True)   # sample from array
rng.shuffle(x)                      # shuffle in-place
```

Why this is preferable?

- ✓ Reproducible: Same seed → same results.
- ✓ Independent streams: Multiple generators without interference.
- ✓ Thread-safe and process-safe.
- ✓ Does not affect global random state used by other libraries / code.

Good practice

Always create your own Generator for projects instead of relying on np.random.* global functions.

Tip: Prefer deterministic functions for reproducible data and random generators for controlled randomness!

NumPy Indexing & Slicing

Important! ⭐

Indexing & Slicing are used to select elements or groups of elements from NumPy arrays.

① **1D indexing**

```
a = np.array([10, 20, 30, 40, 50])
```

index →	0	1	2	3	4
	10	20	30	40	50

`a[2] = 30`

② **2D indexing**

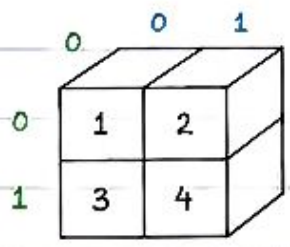
```
b = np.array([[1, 2, 3],
              [4, 5, 6]])
```

	0	1	2
0	1	2	3
1	4	5	6

`b[0,1] = 2`
`b[1,2] = 6`

③ **3D indexing**

```
c = np.array([[[1, 2], [3, 4]],
              [[5, 6], [7, 8]]])
```



`c[0,1,0] = 2`
`c[1,0,1] = 6`

④ **Slicing**

a =	0	1	2	3	4
	10	20	30	40	50

`a[1:4] = [20 30 40]`
(start:stop) stop-1

⑤ **Negative indexing**

index →	0	1	2	3	4
a =	10	20	30	40	50
	-5	-4	-3	-2	-1

`a[-1] = 50`
`a[-3] = 30`

⑥ **Step slicing**

a =	0	1	2	3	4	5	6
	10	20	30	40	50	60	70

`a[::2] = [10 30 50 70]` (step=2)
`a[1::3] = [20 50]`

⑦ **Boolean indexing**

```
a = [5, 12, 8, 15, 3, 20]
mask = a > 10  →  [False, True, False, True, False, True]
```

`a[a > 10] = [12, 15, 20]` ← Example

⑧ **Fancy indexing**

a =	10	20	30	40	50

`a[[0, 2, 4]] = [10, 30, 50]`

⑨ **Integer-array indexing** `idx = np.array([4, 1, 3])`

`a[idx] = [50, 20, 40]`

⑩ **np.where()** Return indices where condition is True.

`np.where(a > 10) → (array([1, 3, 5]),)`

⑪ **np.take()** Take elements at given indices (along axis).

`np.take(a, [1, 3]) → [20, 40]`

⑫ **np.put()** Replace elements at given indices.

`np.put(a, [1, 3], [200, 400])`
`a → [10, 200, 30, 400, 50]`

Understand the Difference ⭐

Expression	Returns	Type	Example (a = [10 20 30 40 50])
a[2]	Single element	Scalar	30
a[2:5]	Slice from index 2 to 4 (stop-1)	1D array (view)	[30 40 50]
a[[2, 5, 7]]*	Elements at indices 2,5,7	1D array (copy)	(if a = [10 20 30 40 50 60 70 80]) [30 60 80]
a[a > 10]	Elements satisfying condition	1D array (copy)	[20 30 40 50] (example a = [5,12,8,15,3,20,25,8])

* Fancy indexing can take any list/array of indices (not necessarily sorted or unique).

Practice More! ☺

NumPy Array Manipulation

Very Important!

1 Array Shape Manipulation

Let,

```
a = np.array([[1, 2, 3]
              [4, 5, 6]])   # shape (2, 3)
```

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

- **reshape()**

  Returns a new array with changed shape.

  `a.reshape(3, 2)` →

1	2
3	4
5	6

  shape (3,2)

- **resize()**

  Resizes array in-place (does not preserve data if new size is smaller).

  ```
  b = a.copy()
  b.resize(3, 2)
  ```

  →

1	2
3	4
5	6

  shape (3,2)

- **ravel()**

  Returns a flattened array (1D view if possible).

  `a.ravel()` → | 1 | 2 | 3 | 4 | 5 | 6 |

  shape (6,)

- **flatten()**

  Returns a flattened copy (1D new array).

  `a.flatten()` → | 1 | 2 | 3 | 4 | 5 | 6 |

  shape (6,)

- **transpose()**

  Permutes dimensions.

  `a.transpose()` →

1	4
2	5
3	6

  shape (3,2)

- **.T**

  Shorthand for transpose().

  `a.T` →

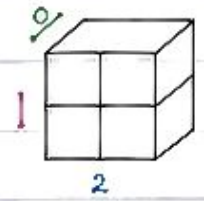
1	4
2	5
3	6

  shape (3,2)

- **swapaxes(axis1, axis2)**

  Swaps two axes.

  ```
  x = np.arange(24).reshape(2, 3, 4)
  x.swapaxes(0, 2)   # shape (4,3,2)
  ```

- **moveaxis(src, dst)**

  Moves an axis to a new position.

  ```
  x.moveaxis(0, 2)   # move axis 0 to position 2
                     # shape (3,4,2)
  ```

- **expand_dims(a, axis)**

  Inserts a new axis.

  ```
  a_exp = np.expand_dims(a, axis=0)
  ```

  → 

  shape (1,2,3)

- **squeeze()**

  Removes single-dimensional entries.

  ```
  s = np.array([[[1, 2, 3]]])   # shape (1,1,3)
  s.squeeze() → [1 2 3]         # shape (3,)
  ```

2 Combining arrays

Let,

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
```

$$a = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad b = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

- **concatanate()**

  Join along existing axis.

  `np.concatanate((a, b), axis=0)`

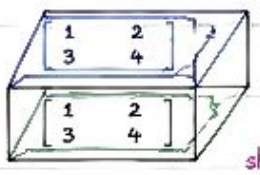
1	2
3	4
5	6
7	8

  shape (4,2)

- **stack()** (new axis)

  Stack along new axis.

  `np.stack((a, b), axis=0)`

  shape (2,2,2)

- **vstack()**

  Stack vertically (axis=0)

  `np.vstack((a, b))`

1	2
3	4
5	6
7	8

  shape (4,2)

- **hstack()**

  Stack horizontally (axis=1)

  `np.hstack((a, b))`

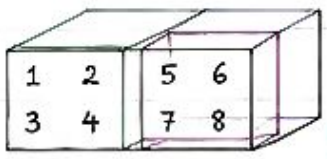
1	2	5	6
3	4	7	8

  shape (2,4)

- **dstack()**

  Stack along depth (axis=2)

  `np.dstack((a, b))`

  shape (2,2,2)

- **column_stack()**

  Stack 1D arrays as columns.

  `np.column_stack((a.ravel(), b.ravel()))`

1	5
2	6
3	7
4	8

  shape (4,2)

- **row_stack()**

  Stack 1D arrays as rows.

  `np.row_stack((a.ravel(), b.ravel()))`

1	2	3	4
5	6	7	8

  shape (2,4)

3 Splitting arrays

```
x = np.arange(1, 13).reshape(3, 4)
```

1	2	3	4
5	6	7	8
9	10	11	12

- **split(ary, indices, axis)**

  Split into equal parts.

  `np.split(x, 2, axis=0)`

1	2	3	4
5	6	7	8

9	10	11	12

- **array_split(ary, indices, axis)**

  Split into equal or unequal parts.

  `np.array_split(x, 3, axis=1)`

1	2
5	6
9	10

3
7
11

4
8
12

- **vsplit(ary, indices)**

  Split vertically (axis=0).

  `np.vsplit(x, 3)`

1	2	3	4

5	6	7	8

9	10	11	12

- **hsplit(ary, indices)**

  Split horizontally (axis=1).

  `np.hsplit(x, 2)`

1	2
5	6
9	10

3	4
7	8
11	12

NumPy Data Types

Important ! ⭐

What are NumPy Data Types?

- NumPy uses fixed-size data types (dtypes) for efficient memory usage and faster computation.
- dtype describes the type of data stored in an array.

Common NumPy Data Types

Type	Alias	Size (bytes)	Range / Meaning
int8	i1	1	-128 to 127
int16	i2	2	-32768 to 32767
int32	i4	4	-2^{31} to $2^{31}-1$
int64	i8	8	-2^{63} to $2^{63}-1$

Data Types in NumPy

- Integer dtypes → int8, int16, int32, int64
- Floating-point dtypes → float16, float32, float64
- Boolean → bool
- Complex numbers → complex64, complex128
- Strings → str_ (unicode strings)
- datetime64 → Date and time values
- timedelta64 → Time differences

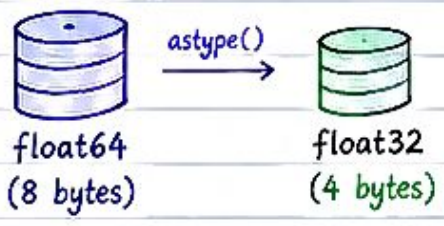
Floating-point dtypes

Type	Size	Precision (≈ decimal digits)
float16	2	3
float32	4	7
float64	8	15

Type Conversion

- Convert array to another dtype using astype().

```
a.astype(np.float32)
```



- Returns a new array with the requested dtype.

Other Data Types

- bool: True / False
- complex: a + bj
- str: text data
- datetime64: date & time (e.g., 2024-01-01)
- timedelta64: time duration (e.g., 3 days)

Precision

- Number of significant digits a floating-point number can represent.

```
float32 → ~7 digits
float64 → ~15 digits
```

Overflow

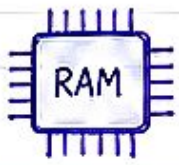
- Happens when a value exceeds the maximum representable value.
- Example: np.int8(130) → 130 % 256 = 2

Underflow

- Happens when a value is smaller than the minimum positive value (very close to 0).
- Example: np.float32(1e-50) → 0.0

Memory Usage

- Smaller dtype → less memory usage.
- Larger dtype → more memory usage.



Memory Usage Example

Array of 1 million elements:

dtype	Item size	Total memory
float64	8 bytes	8 MB
float32	4 bytes	4 MB

Key Concept:

```
a.astype(np.float32)
```

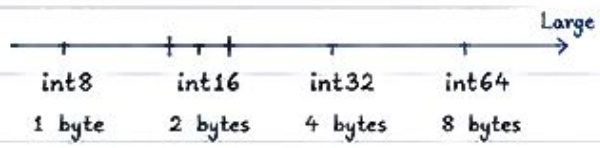
→ Changing the dtype from float64 (8 bytes each) to float32 (4 bytes each) can almost halve the memory usage, which is a huge saving for large arrays.

NumPy Data Types ⭐

Very Important! ⭐

1 Integer dtypes

Signed integers: int8, int16, int32, int64
Unsigned integers: uint8, uint16, uint32, uint64



2 Floating-point dtypes

float16, float32, float64, longdouble

dtype	Size	Precision (approx.)	Range
float16	2 bytes	~3 decimal digits	±6e-8 to ±6e4
float32	4 bytes	~7 decimal digits	±1e-38 to ±3e38
float64	8 bytes	~15 decimal digits	±5e-324 to ±1e308
longdouble	10/16 bytes*	~18+ digits	system dependent

* platform dependent

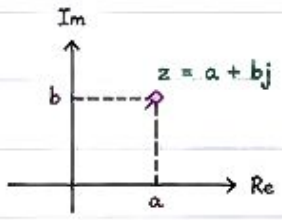
3 Boolean

bool_ (True/False)
Size: 1 byte

Example
```
np.array([True, False, True])
  dtype('bool')
```

4 Complex numbers

complex64 (float32 real + float32 imag)
complex128 (float64 real + float64 imag)



5 Strings

- Fixed-length Unicode strings: dtype='U10' (up to 10 chars)
- Fixed-length bytes strings: dtype='S10' (up to 10 bytes)

↓ Note: Length is fixed. Longer data gets truncated.

Example
```
np.array(['hi', 'hello'], dtype='U5')
  → array(['hi', 'hello'], dtype='<U5')
```

6 datetime64

Represents dates and times.
Example units: 'Y' (year), 'M' (month), 'D' (day),
'h' (hour), 'm' (minute), 's' (second), 'ms', 'us', 'ns'

Example
```
np.array(['2024-01-01', '2024-01-02'],
    print='datetime64[D]')
```

7 timedelta64

Represents time differences (durations).
Same units as datetime64.

Example
```
np.array([10, 20, 30], dtype='timedelta64[s]')
  → 10 sec, 20 sec, 30 sec
```

8 Type conversion

- Implicit conversion (automatic)

  int + float → float

- Explicit conversion

  Use astype() to convert dtype.

```
a = np.array([1, 2, 3], dtype=np.int32)
b = a.astype(np.float32)   # a.astype(np.float32)
```

9 astype()

Returns a copy of array with new dtype.

- Syntax:

```
a.astype(new_dtype, copy=True)
```

Example:

```
a = np.array([1, 2, 3])
a_float = a.astype(np.float32)
a_int8 = a.astype(np.int8)
```

10 Precision

More bytes → More precision
float32 has less precision than float64
(May lose small decimal details)

float32 : 0.12345679
float64 : 0.123456789123456

11 Overflow

Value exceeds maximum representable limit.
Result wraps around.

Example
```
a = np.array([127], dtype=np.int8)
b = a + 1   # 127 + 1 = -128 (overflow!)
```

int8 range: -128 to 127

12 Underflow

Value is too small (close to 0) to be represented.
Result becomes 0.

Example
```
a = np.array([1e-50], dtype=np.float32)
b = a / 1e50   # becomes 0.0 (underflow)
```

13 Memory usage (Why dtype matters!)

Same data, different dtype → different memory usage

dtype	Itemsize	Array of 1 million elements	Memory
int8	1 byte	1,000,000	~0.95 MB
int32	4 bytes	1,000,000	~3.81 MB
int64	8 bytes	1,000,000	~7.63 MB
float32	4 bytes	1,000,000	~3.81 MB
float64	8 bytes	1,000,000	~7.63 MB

Changing dtype can save or waste a lot of memory!

⭐ Important Takeaway

```
a = np.array([1, 2, 3, 4])
print(a.dtype)      # int64 (default)
print(a.nbytes)     # 32 bytes (8 * 4)
```

↓ Convert to float32

```
b = a.astype(np.float32)
print(b.dtype)      # float32
print(b.nbytes)     # 16 bytes (4 * 4)
```

Memory Comparison (1 million integers)



Use the smallest dtype that can hold your data.
Better memory usage, better performance!

Remember
- ✓ Choose right dtype
- ✓ Save memory
- ✓ Avoid overflow
- ✓ Improve speed

NumPy Vectorization

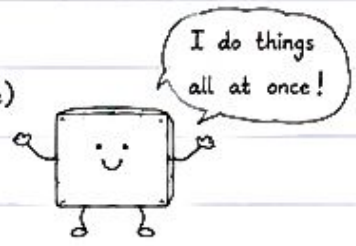
Very Important!

1 What is Vectorization?

Vectorization means performing operations on entire arrays at once, without writing explicit Python loops.

Why it matters?

- ✓ Faster (implemented in optimized C code)
- ✓ More concise and readable
- ✓ Less error-prone
- ✓ Better memory efficiency



Always prefer: `a * 2` over Instead of:

```
for x in a:
    ...
```

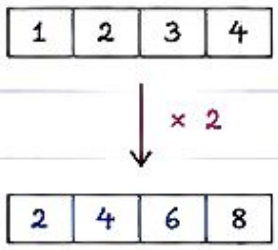
2 Element-wise Operations

Operations are applied element-by-element.

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])
```

a + b	=	[11, 22, 33, 44]
a - b	=	[-9, -18, -27, -36]
a * b	=	[10, 40, 90, 160]
a / b	=	[0.1, 0.1, 0.1, 0.1]

No loops needed!



3 Universal Functions (ufuncs)

NumPy provides fast element-wise functions called ufuncs. They operate on ndarrays.

Examples:

- **Mathematical**
  `np.sqrt()`, `np.exp()`, `np.log()`, `np.sin()`, `np.cos()`, `np.abs()`, ...
- **Arithmetic**
  `np.add()`, `np.subtract()`, `np.multiply()`, `np.divide()`, ...
  `np.power()`, ...
- **Others**
  `np.maximum()`, `np.minimum()`, `np.floor()`, `np.ceil()`, ...

ufuncs are vectorized by default!

4 Broadcasting

Allows operations on arrays of different shapes.
Smaller array "stretches" to match the larger.

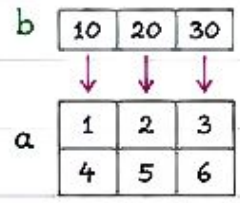
Example 1:

```
a = np.array([1, 2, 3, 4])  # shape (4,)
b = 10                      # scalar
a + b → [11, 12, 13, 14]
```

10 is broadcast to all elements of a

Example 2:

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])   # shape (2,3)
b = np.array([10, 20, 30])  # shape (3,)
a + b → [[11, 22, 33],
         [14, 25, 36]]
```



5 Avoiding Unnecessary Python Loops

Loops run in Python (slow). Vectorized ops run in C (fast).

Using loops (Slow)

```
import numpy as np
a = np.arange(1_000_000)
b = np.zeros_like(a)
for i in range(len(a)):
    b[i] = a[i] * 2
```

☹ Slow in Python!

Vectorized (Fast)

```
import numpy as np
a = np.arange(1_000_000)
b = a * 2
```

☺ Much faster!

6 Master These ufuncs

Function	Name	Example	For a = np.array([0, 1, 2, 3])
np.add(x, y)	Addition	np.add(a, 2)	[2, 3, 4, 5]
np.subtract(x, y)	Subtraction	np.subtract(a, 1)	[-1, 0, 1, 2]
np.multiply(x, y)	Multiplication	np.multiply(a, 3)	[0, 3, 6, 9]
np.divide(x, y)	Division	np.divide(a, 2)	[0. , 0.5, 1. , 1.5]
np.power(x, y)	Power	np.power(a, 2)	[0, 1, 4, 9]
np.sqrt(x)	Square root	np.sqrt(a)	[0. , 1. , 1.414, 1.732]
np.exp(x)	Exponential (e^x)	np.exp(a)	[1., 2.718, 7.389, 20.086]
np.log(x)	Natural log (ln x)	np.log(a+1)	[0., 0.693, 1.099, 1.386]
np.abs(x)	Absolute value	np.abs(np.array([-3, -1, 0, 2]))	[3, 1, 0, 2]
np.sin(x)	Sine (radians)	np.sin(a)	[0. , 0.841, 0.909, 0.141]
np.cos(x)	Cosine (radians)	np.cos(a)	[1. , 0.540, -0.416, -0.990]

These are fast, element-wise and support broadcasting!

7 Small Performance Example

Let's compare on a large array.
(Results will vary on your machine)

Task	Method	Time (approx.)
Square each element	for loop	~ 300 ms
	vectorized (a**2)	~ 3 ms
Multiply by 5	for loop	~ 250 ms
	vectorized (a*5)	~ 2 ms

Vectorization can be 50x - 100x faster!

☺

8 Key Takeaways

- ✓ Use vectorized operations by default.
- ✓ NumPy ufuncs are your best friends.
- ✓ Broadcasting makes your code powerful and concise.
- ✓ Avoid Python loops on large data.

Remember

Write less code.
Run faster.
Think in arrays, not loops!

☺

Example in practice

```
a = np.array([1, 2, 3, 4])
```

- a * 2 → [2, 4, 6, 8]
- np.sqrt(a) → [1. , 1.414, 1.732, 2.]
- np.sin(a) → [0.841, 0.909, 0.141, -0.757]
- np.log(a+1) → [0.693, 1.099, 1.386, 1.609]

NumPy Broadcasting ⭐

Very Important!! ⭐

① What is Broadcasting?

Broadcasting allows NumPy to perform operations on arrays of different shapes.
It "stretches" the smaller array to match the larger one (virtually, without copying data).

Example :

a (3,)

1
2
3

\+

b (3,3)

10	20	30
10	20	30
10	20	30

→

result (3,3)

11	21	31
12	22	32
13	23	33

a is broadcast to shape (3,3) and added element-wise.

② Broadcasting Rules (Align shapes from the right)

1. Compare dimensions from right to left.
2. Dimensions are compatible if :
   - they are equal, or
   - one of them is 1.
3. If one array has fewer dimensions, prepend 1s to its shape.
4. If any pair of dimensions is incompatible, broadcasting is not possible.

If all dimensions are compatible, the result shape is the element-wise maximum of each dimension. ☺

③ Scalar Broadcasting

A scalar (shape : ()) is compatible with any shape.
The scalar is stretched to every element.

Example :

```
a = np.array([[1, 2, 3], [4, 5, 6]])   # shape (2,3)
b = 10                                 # shape () scalar
a + b →  [[11, 12, 13],
          [14, 15, 16]]
```

Scalar acts like an array filled with that value. ☺

④ 1D → 2D Broadcasting

```
a = np.array([1, 2, 3])      # (3,)
b = np.array([[10],
              [20]])         # (2,1)
a + b →
```

a (1,3)

1	2	3

\+

b (2,1)

10
20

↓

result (2,3)

11	12	13
21	22	23

a is broadcast down the rows, b is broadcast across columns.

⑤ 2D → 3D Broadcasting

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])          # (2,3)
b = np.array([[[10, 20, 30]]])     # (1,1,3)
a + b →  result shape (2,1,3)
```

a (2,3)

1	2	3
4	5	6

\+

b (1,1,3)

[[10 20 30]]

↓

result (2,1,3)

slice 0 ↑	11	22	33
slice 1 ↑	41	52	63

b is stretched to (2,1,3) to match a.

⑥ Compatible vs Incompatible Shapes

Compatible ✓

- (3,) and (3,1) → (3,3)
- (2,1,5) and (1,4,5) → (2,4,5)
- (1,) and (4,3,1) → (4,3,1)
- () and any shape → same shape

Incompatible ✗

- (2,3) and (3,2)
- (2,) and (2,2)
- (2,3,4) and (3,4)
- (3,) and (2,3,4)

If any dimension pair (from right) is neither equal nor 1 → Error ! ☹

⑦ np.newaxis / None

np.newaxis and None add a new axis (dimension) at the given position.

Example :

```
x = np.array([1, 2, 3])   # shape (3,)
```

x[:, np.newaxis]
shape → (3,1)

1
2
3

x[np.newaxis, :]
shape → (1,3)

1	2	3

x[:, np.newaxis, :]
shape → (3,1,3)

[1	2	3]
[1	2	3]
[1	2	3]

Use np.newaxis (or None) to align shapes for broadcasting. ☺

⑧ expand_dims()

Adds a new axis at the specified position.

```
x = np.array([1, 2, 3])      # (3,)
```

np.expand_dims(x, axis=0)
shape → (1,3)

1	2	3

np.expand_dims(x, axis=1)
shape → (3,1)

1
2
3

np.expand_dims(x, axis=2)
shape → (3,1,1)

[1]
[2]
[3]

expand_dims is very useful to make shapes compatible for broadcasting. ⭐

⑨ Quick Summary

- ✓ Broadcasting compares shapes from right.
- ✓ Dimensions must be equal or 1 (or missing → 1).
- ✓ Smaller arrays are virtually "stretched".
- ✓ Result shape = element-wise maximum.
- ✓ Use np.newaxis / None / expand_dims() to add dimensions and make shapes compatible.

Example

```
a = np.array([1, 2, 3])          # (3,)
b = np.array([[10], [20]])       # (2,1)
c = np.array([[[100, 200, 300]]])# (1,1,3)
a + b + c →
Result shape (2,1,3)
```

slice 0 ↑	111	212	313
	121	222	323
slice 1 ↑	131	232	333
	131	232	333

All arrays are broadcast to (2,1,3) before the operation. ☺

NumPy Aggregation & Statistics ⭐

Very Important !! ⭐

① What is Aggregation?

Aggregation = combining many values to produce a single summary value.

Example :

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
```

→ sum → 21
→ mean → 3.5
→ max → 6
→ min → 1

② Basic Aggregation Functions

Function	What it does	Example (a = [[1,2,3],[4,5,6]])
sum()	Sum of all elements	a.sum() → 21
mean()	Average of elements	a.mean() → 3.5
median()	Median (middle value)	np.median(a) → 3.5
min()	Minimum value	a.min() → 1
max()	Maximum value	a.max() → 6
std()	Standard deviation	a.std() → 1.7078
var()	Variance	a.var() → 2.9167

③ Index of Min / Max

- argmin() → index of minimum value
- argmax() → index of maximum value

For a = [[1, 2, 3],
[4, 0, 6]]]

```
a.argmin() → (1, 1)    # position of 0
a.argmax() → (1, 2)    # position of 6
```

Returns index in flattened array as a tuple (row_index, col_index). ☺

④ Cumulative Functions

- cumsum() → cumulative sum
- cumprod() → cumulative product

For a = [1, 2, 3, 4]

a.cumsum() → | 1 | 3 | 6 | 10 |

a.cumprod() → | 1 | 2 | 6 | 24 |

Useful for running totals or product calculations. ☺

⑤ Percentile & Quantile

- percentile(x, p) → p^{th} percentile (0 to 100)
- quantile(x, q) → qth quantile (0 to 1)

For a = [1, 2, 3, 4, 5, 6, 7, 8, 9]

```
np.percentile(a, 50) → 5.0    # median
np.percentile(a, 90) → 8.2
```

```
np.quantile(a, 0.25) → 2.0
np.quantile(a, 0.75) → 7.0
```

quantile(q) is same as percentile(q * 100) ⭐

⑥ MOST IMPORTANT : axis

Axis tells NumPy along which direction to aggregate.

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
shape → ((2, 3])
```

rows (axis=0) ↓

1	2	3
4	5	6

columns (axis=1) >>

axis = 0 (Column-wise)

Aggregate down the rows (per column).

Result shape → (3,)

```
a.sum(axis=0)  → [ 5   7   9 ]   # 1+4  2+5  3+6
a.mean(axis=0) → [2.5 3.5 4.5]
a.min(axis=0)  → [ 1   2   3 ]
a.max(axis=0)  → [ 4   5   6 ]
a.std(axis=0)  → [1.5 1.5 1.5]
```

One value per column. ☺

axis = 1 (Row-wise)

Aggregate across columns (per row).

Result shape → (2,)

```
a.sum(axis=1)  → [ 6   15 ]   # 1+2+3  4+5+6
a.mean(axis=1) → [ 2.   5. ]
a.min(axis=1)  → [ 1    4 ]
a.max(axis=1)  → [ 3    6 ]
a.std(axis=1)  → [0.816 0.816]
```

One value per row. ☺

axis = None (Default)

Aggregate over the entire array.

Result is a scalar.

```
a.sum()  → 21
a.mean() → 3.5
a.min()  → 1
a.max()  → 6
a.std()  → 1.7078
```

Single summary value. ☺

⑦ For 3D Array

```
b = np.array(
    [[[1, 2], [3, 4]],
     [[5, 6], [7, 8]]])
shape → (2, 2, 2)
```

1	2
3	4
5	6
7	8

axis=0 → depth (front to back)
axis=1 → rows (top to bottom)
axis=2 → columns (left to right)

Operation	axis = 0 (result shape)	axis = 1 (result shape)	axis = 2 (result shape)
sum	[[6, 8], [10, 12]] (2, 2)	[[4, 6], [12, 14]] (2, 2)	[[3, 7], [11, 15]] (2, 2)
mean	[[3, 4], [5, 6]] (2, 2)	[[2., 3.], [6., 7.]] (2, 2)	[[1.5, 3.5], [5.5, 7.5]] (2, 2)
max	[[5, 6], [7, 8]] (2, 2)	[[3, 4], [7, 8]] (2, 2)	[[2, 4], [6, 8]] (2, 2)
min	[[1, 2], [3, 4]] (2, 2)	[[1, 2], [5, 6]] (2, 2)	[[1, 2], [5, 6]] (2, 2)

⑧ Quick Summary

- ✓ Aggregation reduces many values to a single summary.
- ✓ axis = 0 → column-wise (down)
- ✓ axis = 1 → row-wise (across)
- ✓ axis = None → whole array
- ✓ argmin/argmax give index of min/max element.
- ✓ cumsum/cumprod for running totals/products.
- ✓ percentile/quantile for position-based statistics.

Practice with different axes until it feels natural! ⭐

⑨ Cheat Sheet (Remember!)

sum() mean() median() min() max() std() var()

argmin() argmax() cumsum() cumprod() percentile() quantile()

→ Don't forget axis ! ☺

NumPy Searching & Sorting ⭐

Very Important !! ⭐

Searching → Find index/position of elements that satisfy a condition or where to insert.

Sorting → Arrange elements in ascending order (by default).

Key Points

- Most functions work along a given axis (default axis = -1).
- They do not change the original array (except where noted).
- Always try to avoid full sorting for large data when you just need top-k or positions.

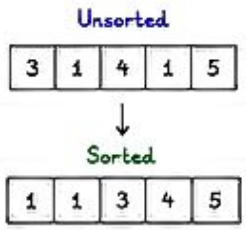
😀

1 sort()

- Returns a sorted copy of the array.
- Ascending by default.
- Syntax : `np.sort(a, axis=-1, kind='quicksort')`

Example :

```
a = np.array([3, 1, 4, 1, 5])
np.sort(a) → 1 1 3 4 5
```



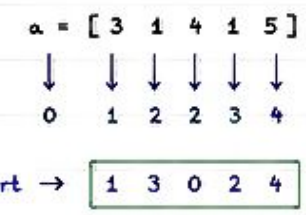
2 argsort()

- Returns the indices that would sort the array.
- Useful for ranking or ordering.
- Syntax : `np.argsort(a, axis=-1, kind='quicksort')`

Example :

```
a = np.array([3, 1, 4, 1, 5])
np.argsort(a) → 1 3 0 2 4
```

(indices of sorted order)



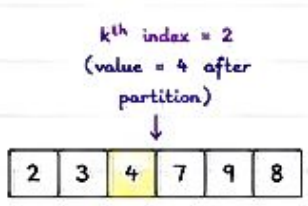
3 partition()

- Partially sorts the array around the kth index.
- Elements smaller than k^{th} are on the left, greater on the right. Order within sides is not sorted.
- Syntax : `np.partition(a, kth)`

Example :

```
a = np.array([7, 2, 9, 4, 3, 8]) , kth = 2
np.partition(a, 2) → 2 3 4 7 9 8
```

<= 4 (kth value) >= 4



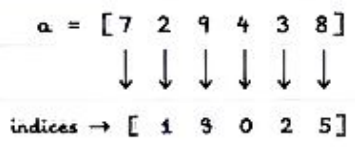
4 argpartition()

- Returns indices that partition the array.
- Faster than argsort() when you need only top-k elements.
- Syntax : `np.argpartition(a, kth)`

Example :

```
a = np.array([7, 2, 9, 4, 3, 8]) , kth = 2
np.argpartition(a, 2) → 1 4 3 0 2 5
```

first 3 indices → positions of 3 smallest elements



5 searchsorted()

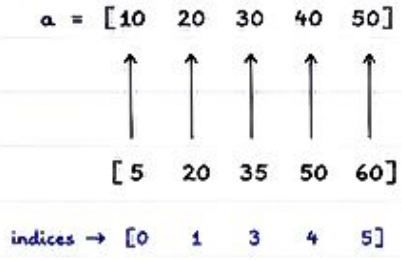
- Returns indices where elements should be inserted to maintain order.
- Syntax : `np.searchsorted(a, v, side='left')`

Example :

```
a = np.array([10, 20, 30, 40, 50])
v = [5, 20, 35, 50, 60]
np.searchsorted(a, v) → 0 1 3 4 5
```

Meaning :

- 5 → index 0
- 20 → index 1
- 35 → index 3 ...

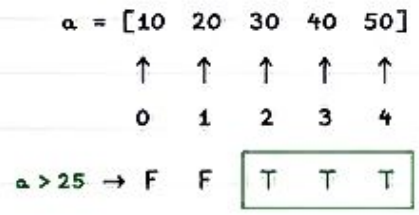


6 where()

- Returns indices where a condition is True.
- Syntax : `np.where(condition)`
- Example :

```
a = np.array([10, 20, 30, 40, 50])
np.where(a > 25) → (array([2, 3, 4]),)
```

(elements > 25 are at indices 2, 3, 4)

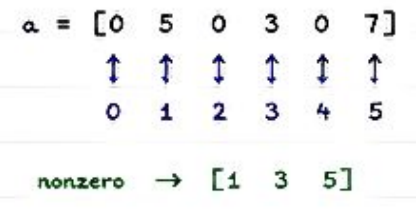


7 nonzero()

- Returns indices of non-zero elements.
- Equivalent to where(a != 0).
- Syntax : `np.nonzero(a)`

Example :

```
a = np.array([0, 5, 0, 3, 0, 7])
np.nonzero(a) → (array([1, 3, 5]),)
```



8 any()

- Returns True if any element (or any along axis) is True.
- Syntax : `np.any(a, axis=None)`
- Example :

```
a = np.array([False, False, True, False])
np.any(a) → True
np.any(a == True) → True
np.any(a == True) along axis in 2D also works.
```

Useful with boolean arrays! 😀

9 all()

- Returns True if all elements (or all along axis) are True.
- Syntax : `np.all(a, axis=None)`
- Example :

```
a = np.array([True, True, True])
np.all(a) → True
b = np.array([True, False, True])
np.all(b) → False
```

Checks if every condition is satisfied ! 😀

10 PRACTICAL APPLICATION

⭐ Find the top 10 values without fully sorting a huge array.

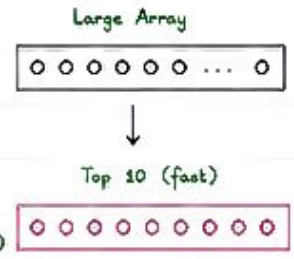
Problem : Get top 10 largest values from a very large array.

Solution using argpartition() (Fast)

① Use argpartition to find indices of top 10 values.
② Take those indices and sort only those values (optional).

Example :

```
a = np.random.randint(0, 1000000, size=100000)  # huge array
k = 10
top10_idx = np.argpartition(a, -k)[-k:]  # indices of top 10 (unordered)
top10 = a[top10_idx]                     # top 10 values (unordered)
top10_sorted = np.sort(top10)[::-1]      # descending order
print(top10_sorted)
```



Summary

- ☑ Use sort/argsort to get fully sorted results.
- ☑ Use partition/argpartition for top-k problems.
- ☑ Use searchsorted for efficient insert positions.
- ☑ Use where/nonzero for condition based indexing.
- ☑ Use any/all for boolean checks.

Practice more, Code better ! ⭐

NumPy Missing / Invalid Values

Very Important !! ☆

1 Special Values in NumPy

- NumPy provides special floating point values to represent missing or invalid data.

np.nan	→	Not a Number (NaN)
np.inf	→	Positive Infinity
-np.inf	→	Negative Infinity

These are float values, not numbers! ☺

2 Representation

- np.nan is a "Not a Number" value.
- np.inf and -np.inf represent infinity.
- They are of type float.

Example :

```
import numpy as np
a = np.array([1.0, np.nan, np.inf, -np.inf])
print(a)    → [ 1.   nan   inf   -inf ]
```

3 np.isnan()

- Checks element-wise for NaN.
- Returns a boolean array.

Example :

```
a = np.array([1.0, np.nan, 2.5, np.nan])
np.isnan(a) → [False True False True]
```

index →	0	1	2	3
a	1.0	nan	2.5	nan
isnan	False	True	False	True

4 np.isinf()

- Checks element-wise for +inf or -inf.
- Returns a boolean array.

Example :

```
a = np.array([1.0, np.inf, -np.inf, 5.0])
np.isinf(a) → [False True True False]
```

index →	0	1	2	3
a	1.0	inf	-inf	5.0
isinf	False	True	True	False

5 np.isfinite()

- Checks for finite (neither NaN nor inf).
- Returns a boolean array.

Example :

```
a = np.array([1.0, np.nan, np.inf, -np.inf, 5.0])
np.isfinite(a)
    → [True False False False True]
```

index →	0	1	2	3	4
a	1.0	nan	inf	-inf	5.0
isfinite	True	False	False	False	True

6 np.nan_to_num()

- Replaces NaN, +inf and -inf with finite numbers.
- Useful for cleaning arrays before computations.

Great for model training & ML pipelines! ☆

Syntax : `np.nan_to_num(a, nan=0.0, posinf=None, neginf=None)`

Example :

```
a = np.array([1.0, np.nan, np.inf, -np.inf, -5.0])
np.nan_to_num(a) → [ 1.   0.   0.   0.  -5. ]
```

Custom replacement :

```
np.nan_to_num(a, nan=-1, posinf=999, neginf=-999)
    → [ 1.   -1.   999. -999.  -5. ]
```

7 NaN-aware Aggregations

- NumPy has special functions that ignore NaN values while computing results.

Function	What it does	Example (a = [1,np.nan,3, np.nan, 5])	Result
np.nansum()	Sum ignoring NaN	np.nansum(a)	9
np.nanmean()	Mean ignoring NaN	np.nanmean(a)	3.0
np.nanmedian()	Median ignoring NaN	np.nanmedian(a)	3.0
np.nanmin()	Min ignoring NaN	np.nanmin(a)	1
np.nanmax()	Max ignoring NaN	np.nanmax(a)	5
np.nanstd()	Std dev ignoring NaN	np.nanstd(a)	≈ 1.633

Regular functions (sum, mean, etc.) will return NaN if any NaN is present. ☺

8 Understand the Difference

Type	What it is	Where used	Example	Type	Equality Check	Notes
None	Python's null object	Used in Python objects, lists, dictionaries, etc.	x = None	<class 'NoneType'>	x is None → True x == None → True	Not a numeric value. Cannot be used in NumPy numeric arrays.
np.nan	NumPy's "Not a Number" (NaN)	Used in NumPy arrays (float dtype)	x = np.nan	<class 'float'>	np.nan == np.nan → False np.isnan(x) → True	NaN is not equal to anything, not even itself!
NaN	Just another name for np.nan	Same as np.nan	x = float('nan') or x = np.nan	<class 'float'>	Same as np.nan	Used in data science to denote missing numeric data.
NaT	Not-a-Time	Used in NumPy datetime64[ns] arrays	x = np.datetime64('NaT')	dtype('datetime64[ns]')	np.isnat(x) → True	Represents missing datetime value.
inf / -inf	Positive / Negative Infinity	Used in NumPy arrays (float dtype)	x = np.inf y = -np.inf	<class 'float'>	np.isinf(x) → True np.isinf(y) → True	Not missing data, but extreme values. Handle carefully!

Quick Tips

- ☑ Use np.isnan() to find missing values.
- ☑ Use nan-aware functions for clean stats.
- ☑ Use np.nan_to_num() before ML models.
- ☑ np.isfinite() is helpful to filter valid data.
- ☑ Always be aware of the data type you are working with!

☺

NumPy Linear Algebra

Advanced NumPy!

① Vectors

- 1-D arrays are called vectors.
- Order (n,)

Example :

```
v = np.array([1, 2, 3])
```

Shape : (3,)

```
1
2
3
```

② Matrices

- 2-D arrays are called matrices.
- Order (m, n)

Example :

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])
```

Shape : (2, 3)

```
1  2  3
4  5  6
```

Useful Notation

- Scalars : 5
- Vectors : (n,)
- Matrices : (m, n)
- Square Matrix : (n, n)

③ Dot Product

- For vectors of same length.
- Result is a scalar.

Example :

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
np.dot(a, b)  → 32
```

1×4 + 2×5 + 3×6
= 4 + 10 + 18
= 32

④ Matrix Multiplication

- For matrices A (m×n) and B (n×p)
- Result is (m×p) matrix.

Example :

```
A = [1, 2],      B = [5, 6],
    [3, 4])          [7, 8]

A @ B →  [19, 22],
         [43, 50]]
```

⑤ @ Operator

- Matrix multiplication operator in Python 3.5+
- Same as np.matmul()

Example :

```
A @ B  ==  np.matmul(A, B)
A @ B  ==  np.dot(A, B)
           (for 2-D arrays)
```

⑥ np.dot() vs np.matmul()

Function	Works With	Notes
np.dot(a, b)	Vectors, Matrices, Higher-D arrays	Behavior changes with dimensions.
np.matmul(a, b)	Matrix, Stack of Matrices	Better for linear algebra operations.
a @ b	Same as np.matmul()	Most readable!

⑦ Transpose

- Swap rows and columns.

Example :

```
A = [[1, 2, 3],
     [4, 5, 6]]

A.T →  [1, 4],
       [2, 5])
       [3, 6]]
```

- For vectors v.T is same as v

⑧ Inverse

- Only for square matrices.
- A @ A.inv = I (Identity)

Example :

```
A = [ 4,  7 ]
    [ 2,  6 ]

np.linalg.inv(A) →  0.6   -0.7 ]
                    -0.2   0.4 ]
```

⑨ Determinant

- Scalar value that gives properties of matrix.

Example :

```
A = [ 1,  2 ]
    [ 3,  4 ]

np.linalg.det(A) → -2.0
```

⑩ Rank

- Maximum number of linearly
- independent rows/columns.

Example :

```
A = [1, 2, 3],
    [2, 4, 6]
    [1, 1, 1]

np.linalg.matrix_rank(A) → 2
```

⑪ Norm

- Measure of size/length.
- For vector → length
- For matrix → magnitude

Example :

```
v = [3, 4]

np.linalg.norm(v)  → 5.0
np.linalg.norm(v, ord=1) → 7.0
```

⑫ Eigenvalues & Eigenvectors

- For square matrix A:
- A v = λ v

Example :

```
A = [ 2,  0],
    [ 0,  3]

vals, vecs = np.linalg.eig(A)

Eigenvalues  → [2.  3.]

Eigenvectors → [1.  0.]
               [0.  1.]]
```

Columns of vecs are eigenvectors.

⑬ Solving Linear Systems

- Solve Ax = b for x
- Use np.linalg.solve(A, b)

Example :

```
A = [ 3,  1 ]      b = [9, 8]
    [ 1,  2 ]

x = np.linalg.solve(A, b)

x →  2.,  3. ]
```

Why use solve?

- ✓ Faster and more stable than computing inverse and multiplying.
- ✓ Avoids numerical errors.

⑭ Important Notes

- All functions are in np.linalg module.
- Most functions require square matrices (n x n).
- Check shapes before operations.
- Use float data type for accurate results.
- For singular matrix, inverse does NOT exist!

Singular Matrix

Determinant = 0
→ Inverse does not exist.

⑮ Master np.linalg Functions

Function	What it does	Example	Notes
np.linalg.inv(A)	Inverse of matrix A	np.linalg.inv(A)	A must be square & non-singular
np.linalg.det(A)	Determinant of A	np.linalg.det(A)	Returns a scalar
np.linalg.eig(A)	Eigenvalues and eigenvectors	eigvals, eigvecs = np.linalg.eig(A)	eigvecs columns are eigenvectors
np.linalg.solve(A, b)	Solve Ax = b	np.linalg.solve(A, b)	A must be square
np.linalg.norm(x)	Matrix or vector norm	np.linalg.norm(x)	ord=2 (default) gives Euclidean norm
np.linalg.matrix_rank(A)	Rank of matrix A	np.linalg.matrix_rank(A)	Rank ≤ min(m, n)
np.linalg.svd(A)	Singular Value Decomposition	U, s, Vt = np.linalg.svd(A)	$A = U \Sigma V^T$
np.linalg.qr(A)	QR Decomposition	Q, R = np.linalg.qr(A)	$A = Q R$, $Q^T Q = I$

SVD & QR are powerful for data science & ML applications!

Quick Summary

- ☑ Use @ or np.matmul() for matrix multiplication.
- ☑ Use np.linalg.solve() instead of inv(A) @ b.
- ☑ Check for singular matrices before using inv().
- ☑ eig() helps in PCA, vibrations, stability analysis.
- ☑ svd() is powerful for data compression & noise reduction.
- ☑ Linear algebra is the backbone of ML & Data Science!

NumPy Advanced

Advanced NumPy!

1 Memory Model

- NumPy arrays store data in a contiguous block of memory.
- Understanding memory helps you write faster and safer code.

Key Attributes

arr.shape	→	size of each dimension
arr.dtype	→	data type
arr.itemsize	→	size of one element (bytes)
arr.nbytes	→	total bytes used
arr.strides	→	steps (in bytes) to move in each dimension
arr.base	→	base object if view

2 Views vs Copies

- A view looks at the same data.
- A copy owns its own memory.

Example :

```
a = np.arange(6)
a = [0 1 2 3 4 5]
```

View

```
b = a[2:5]
b = [2 3 4]
```

b.base is a → True

(changes in a reflect in b)

Copy

```
c = a[2:5].copy()
c = [2 3 4]
```

c.base is a → False

(changes in a do not reflect in c)

Test :

```
np.shares_memory(a, b) → True
np.shares_memory(a, c) → False
```

3 base

- If an array is a view, .base points to the original array.
- If it owns data, .base is None.

Example :

```
a = np.arange(10)
b = a[2:8]
c = b[::2]
```

- c.base → b (direct base)
- b.base → a
- a.base → None

4 Memory Layout

C-order (Row-major)

- Rows are stored contiguously.
- Last index changes fastest.
- Default in NumPy.

Index →	0	1	2
0	a00	a01	a02
1	a10	a11	a02

Memory → a00 a01 a02 a10 a11 a12

Fortran-order (Column-major)

- Columns are stored contiguously.
- First index changes fastest.
- Use order='F'.

Index →	0	1	2
0	a00	a01	a02
1	a10	a11	a12

Memory → a00 a10 a01 a11 a01 a12

5 Contiguous Arrays

- C-contiguous : row-major contiguous.
- F-contiguous : column-major contiguous.
- Check with flags.

Example :

```
a = np.arange(6).reshape(2, 3)
a.flags['C_CONTIGUOUS']  → True
a.flags['F_CONTIGUOUS']  → False

b = np.asfortranarray(a)
b.flags['C_CONTIGUOUS']  → False
b.flags['F_CONTIGUOUS']  → True
```

6 Strides

- Strides show how many bytes to jump to move one step in each dimension.
- Computed automatically by NumPy.

Example :

```
a = np.arange(6).reshape(2, 3)
a  [[0 1 2]
    [3 4 5]]
a.strides → (24, 8)
```

int64 (8 bytes) each element

- 24 ↕ : jump 24 bytes to go down one row
- 8 ↔ : jump 8 bytes to go right one column

7 np.ascontiguousarray()

- Returns a C-contiguous array.
- If input is already C-contiguous, no copy is made.
- Useful for speed in C-API / BLAS calls.

Example :

```
a = np.arange(12)[::2]    # not contiguous
a.flags['C_CONTIGUOUS']  → False
b = np.ascontiguousarray(a)
b.flags['C_CONTIGUOUS']  → True
```

8 Performance

1 Vectorization

- Use array operations instead of Python loops.
- Much faster!

Example :

```
x = np.arange(1_000_000)
y = x * 2 + 5          # fast
```

(Avoid for loops!)

2 Broadcasting

- Operate on arrays of different
- shapes efficiently.

Example :

```
a = np.arange(3)            shape (3,)
b = np.arange(6).reshape(2,3)  shape (2,3)

a + b → [0 2 4]
        [3 5 7]
```

3 Memory Allocation

- np.empty is faster (uninitialized).
- np.zeros initializes to zero.
- np.ones initializes to one.

Example :

- np.empty((1000, 1000))
- np.zeros((1000, 1000))
- np.ones((1000, 1000))

4 Temporary Arrays

- Chained operations create temp arrays (extra memory).
- Use in-place ops when possible.

```
x += 1    (in-place)
```

5 np.einsum()

- Einstein summation.
- Great for complex tensor ops.

Example :

```
np.einsum('ij, jk->ik', A, B)
```

6 np.matmul()

- Matrix multiplication (@ operator).
- Better for 1D/2D arrays.
- np.matmul(A, B) or A @ B

7 np.tensordot()

- Sum over specified axes.
- Generalization of dot product.

```
np.tensordot(A, B, axes=([1],[0]))
```

9 Advanced Operations

1 Structured Arrays

- Heterogeneous data with named fields.

Example :

```
dt = np.dtype([('name','U10'),
               ('age', 'i4'),
               ('height', 'f4')])
a = np.array([('Ram', 20, 175.5),
              ('Sita', 18, 160.2)],
             dtype=dt)
a['age'] → array([20, 18])
```

2 Record Arrays

- Like structured arrays, but field access as attributes.

Example :

```
r = a.view(np.recarray)
r.age  → array([20, 18])
r.name → array(['Ram','Sita'])
```

3 Masked Arrays (np.ma)

- Handle missing or invalid data.

Example :

```
import numpy.ma as ma
x = ma.array([1, 2, -1, 4],
             mask=[0, 0, 1, 0])
x.mean()  → 2.333...
```

4 Polynomial Operations (np.polynomial)

- Easy polynomial math.

Example :

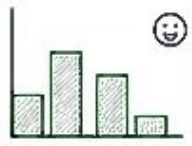
```
from numpy.polynomial import Polynomial
p = Polynomial([1, 2, 3])  # 1 + 2x + 3x^2
p(2) → 17
```

5 FFT (Fast Fourier Transform)

- Convert signal to frequency domain.

Example :

```
x = np.random.rand(8)
X = np.fft.fft(x)
```

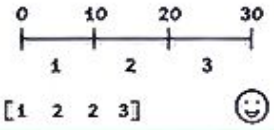


7 Binning (Digitize)

- Assign each value to a bin index.

Example :

```
bins = [0, 10, 20, 30]
x = [5, 12, 17, 25]
np.digitize(x, bins) → [1 2 2 3]
```



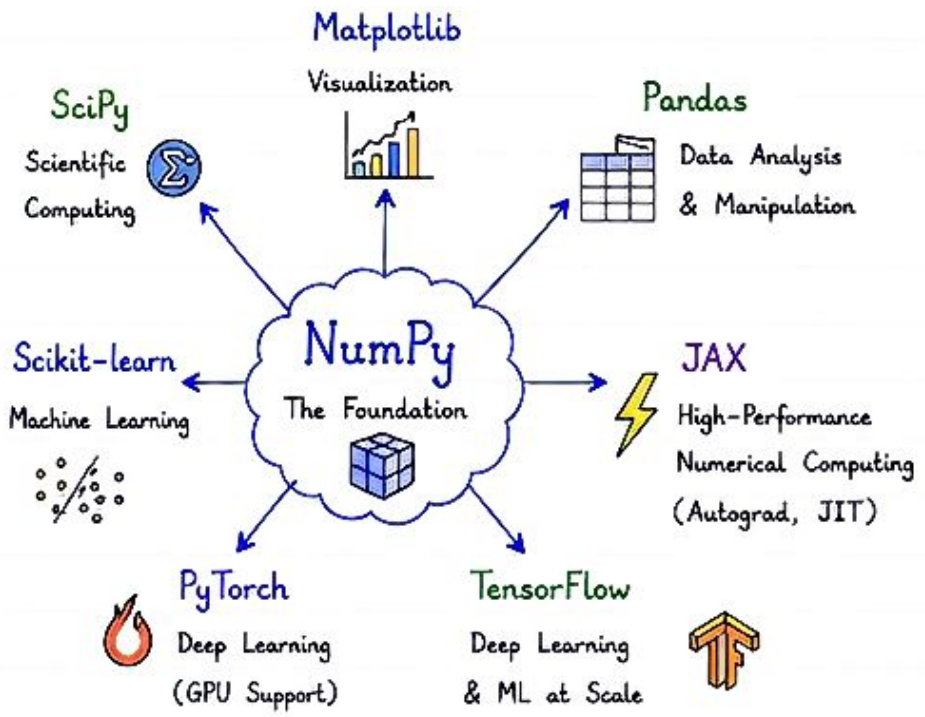
Master memory, operations & tools for high performance with NumPy!

NumPy Advanced Ecosystem ★

NumPy at the heart of Scientific Python! ☺

What is the NumPy Ecosystem?

- NumPy provides the foundation for numerical computing in Python.
- Many powerful libraries are built on top of NumPy ndarrays.
- These libraries help us in scientific computing, data analysis, machine learning, deep learning, visualization and more. ☺



Key Takeaway

Learn NumPy well first! It is the backbone that powers the entire ecosystem. ☺

① SciPy

- Built on NumPy.
- Provides advanced algorithms for science, engineering & math.
- Modules: optimize, integrate, linalg, stats, signal, sparse, etc.

Example:

```
from scipy import optimize
import numpy as np
f = lambda x: (x-2)**2
res = optimize.minimize_scalar(f)
```

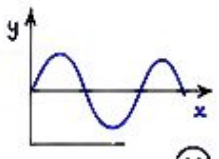
☺

② Matplotlib

- Built on NumPy.
- Used for 2D plots, charts, histograms, scatter plots, etc.
- Works seamlessly with NumPy arrays.

Example:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.plot(x, y)
plt.title("Sine Wave")
plt.show()
```



☺

③ Scikit-learn

- Built on NumPy, SciPy, Matplotlib.
- Simple & efficient tools for machine learning tasks.
- Includes classification, regression, clustering, preprocessing, etc.

Example:

```
from sklearn.linear_model import LinearRegression
import numpy as np
X = np.array([[1], [2], [3], [4]])
y = np.array([2, 4, 6, 8])
model = LinearRegression().fit(X, y)
y_pred = model.predict([[5]])
```

☺

④ Pandas

- Built on NumPy.
- Provides data structures (Series, DataFrame) for data manipulation and analysis.
- Uses NumPy arrays internally.

Example:

```
import pandas as pd
import numpy as np
data = np.array([[1,2], [3,4]])
df = pd.DataFrame(data,
     columns=['A', 'B'])
```

	A	B
0	1	2
1	3	4

☺

⑤ JAX

- Built with NumPy API compatibility.
- High-performance numerical computing.
- Features: Autograd, JIT compilation, Vectorization (vmap).
- Great for research & acceleration.

Example:

```
import jax.numpy as jnp
from jax import grad
def f(x):
    return jnp.sum(x**2)
g = grad(f)
x = jnp.array([1.0, 2.0, 3.0])
print(g(x))
```

☺

⑥ PyTorch

- Built on top of NumPy.
- Popular deep learning framework.
- Supports dynamic computation graphs & GPU acceleration.
- Converts easily between NumPy arrays and Tensors.

Example:

```
import torch
import numpy as np
a = np.array([1,2,3], dtype=np.float32)
t = torch.from_numpy(a)
print(t)
```

NumPy array ⇄ PyTorch Tensor

☺

⑦ TensorFlow

- Built on NumPy.
- End-to-end platform for machine learning & deep learning.
- Uses tensors (similar to ndarrays).

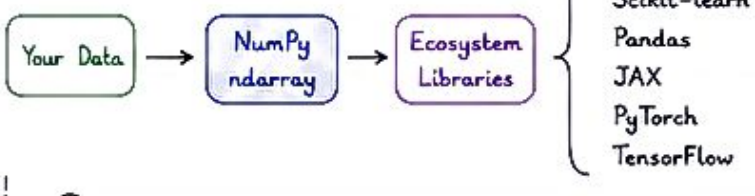
Example:

```
import tensorflow as tf
import numpy as np
a = np.array([1,2,3],
      dtype=np.float32)
t = tf.convert_to_tensor(a)
print(t)
```

☺

NumPy's Role in the Ecosystem

- Provides the ndarray — the core data structure.
- Enables fast numerical operations.
- Serves as the common language for data exchange between all these libraries.
- Master NumPy → Many libraries become easy to learn! ☺



Remember

You don't need to master these libraries now. Just know how NumPy connects everything together! ☺



Pandas – Beginner ⭐

Data Analysis Made Easy! ☺

```
import pandas as pd
# pd is the usual alias
```

What is Pandas?

- Pandas is a Python library used for data manipulation and analysis.
- Built on top of NumPy.
- Provides easy-to-use data structures (Series, DataFrame) and powerful data analysis tools.

Why use Pandas?

- ✓ Easy handling of structured data
- ✓ Data cleaning and preparation
- ✓ Powerful analysis and aggregation
- ✓ Works well with NumPy, Matplotlib, Scikit-learn and more. ☺

① Series

- A Series is a one-dimensional labeled array capable of holding any data type.
- It has two parts:
  - Values
  - Index (labels)

Example:

```
s = pd.Series([10, 20, 30, 40],
              index=['a', 'b', 'c', 'd'])
```

s (Values)

Index	
a	10
b	20
c	30
d	40

One Dimensional (1D)

Access: `s['a']` → 10

② DataFrame

- A DataFrame is a two-dimensional labeled data structure with columns of potentially different data types.
- It is like a table (rows x columns).

Example:

```
df = pd.DataFrame({
    'Name'  : ['Alice', 'Bob', 'Charlie'],
    'Age'   : [25, 30, 22],
    'Score' : [88.5, 92.0, 79.0]
})
```

Index	Name	Age	Score
0	Alice	25	88.5
1	Bob	30	92.0
2	Charlie	22	79.0

Two Dimensional (2D)

Columns

③ Index

- Index is the set of labels for rows (in DataFrame) or for values (in Series).
- By default, it starts from 0.
- We can set our own index.

Example:

```
df = pd.DataFrame({'A':[10, 20, 30]},
                  index=['x', 'y', 'z'])
```

Index	A
x	10
y	20
z	30

Access: `df.loc['x']` → first row

④ Columns

- Columns are the vertical labels in a DataFrame.
- Each column can hold a different data type.

Example:

```
df = pd.DataFrame({
    'Name': ['Ravi', 'Neha', 'Arjun'],
    'Age' : [21, 22, 23],
    'City' : ['Hyd', 'Pune', 'Delhi']
})
```

	Name	Age	City
0	Ravi	21	Hyd
1	Neha	22	Pune
2	Arjun	23	Delhi

Columns

Access column: `df['Age']`

⑤ Rows

- Rows are the horizontal entries in a DataFrame.
- Each row represents an observation/record.

Example:

(Same DataFrame)

	Name	Age	City	
0	Ravi	21	Hyd	→ Row 0
1	Neha	22	Pune	→ Row 1
2	Arjun	23	Delhi	→ Row 2

Access row:

`df.loc[1]` → second row (Neha, 22, Pune)

`df.iloc[2]` → third row (Arjun, 23, Delhi)

⑥ Data Types

- Pandas supports different data types for columns.
- You can check data types using .dtypes
- Common data types:

Type	Description
int64	Integer numbers
float64	Decimal numbers
object	Strings / mixed data
bool	True / False values
datetime64[ns]	Date and time
category	Categorical data
timedelta[ns]	Time differences

Example:

`df.dtypes` →

```
Name      object
Age        int64
City      object
dtype: object
```

Summary

- **Pandas** (Data Analysis with Python)
  - **Series** — 1D labeled data
    - **Index** — labels for values
  - **DataFrame** — 2D labeled data (table)
    - **Columns** — vertical labels
    - **Rows** — horizontal entries

Data Types — define the kind of data stored

Remember

- ✓ Pandas makes data easy to clean, explore and analyze.
- ✓ Start small, practice daily, and you will get better! ☺

Pandas Series

Series = One-dimensional labeled array ☺

```
import pandas as pd
```

What is Series?

- A Series is a 1D labeled array
- Consists of values and an index.
- All elements can be of any data type.

Series

index	values
0	10
1	20
2	30
3	40

① Creating Series

- From list / array

```
s = pd.Series([10, 20, 30, 40])
```

index	values
0	10
1	20
2	30
3	40

- From dictionary

```
s = pd.Series({'a':10, 'b':20, 'c':30})
```

index	values
a	10
b	20
c	30

② Index

- Default index is 0 to n-1
- Access with s.index

```
s = pd.Series([100, 200, 300])
s.index
```

↓

RangeIndex(start=0, stop=3, step=1)

③ Custom Index

- Provide your own index labels

```
s = pd.Series([10, 20, 30], index=['x', 'y', 'z'])
```

x	10
y	20
z	30

s.index → Index(['x', 'y', 'z'], dtype='object')

④ Accessing Values

- Using index label (default)

```
s = pd.Series([10, 20, 30],
              index=['a', 'b', 'c'])
s['a'] → 10
s['b'] → 20
```

- Using position (integer location)

```
s[0] → 10
s[1] → 20
s[2] → 30
```

Tip: s[label] uses label
s[position] uses position

⑤ loc (label-based)

- Access using index labels

```
s = pd.Series([10,20,30,40],
              index=['w','x','y','z'])
s.iloc['x'] → 20
s.iloc[['w','z']]
```

w	10
z	40

- Slicing with labels (inclusive)

```
s.iloc['w':'y']
```

w	10
x	20
y	30

⑥ iloc (position-based)

- Access using integer position

```
s = pd.Series([10,20,30,40],
              index=['w','x','y','z'])
s.iloc[0] → 10
s.iloc[2] → 30
s.iloc[[0,3]]
```

w	10
z	40

- Slicing with positions (end exclusive)

```
s.iloc[1:3]
```

x	20
y	30

⑦ Boolean Filtering

- Filter values using a condition

```
s = pd.Series([10, -5, 20, -1, 30],
              index=list('abcde'))
s
```

a	10
b	-5
c	20
d	-1
e	30

s[s > 0] →

a	10
c	20
e	30

⑧ Arithmetic

- Element-wise operations

```
s1 = pd.Series([10,20,30], index=['a','b','c'])
s2 = pd.Series([1, 2, 3], index=['a','b','c'])
```

s1 + s2

a	11
b	22
c	33

s1 - s2

a	9
b	18
c	27

s * 2

a	20
b	40
c	60

s1 / 10

a	1.0
b	2.0
c	3.0

⑨ Missing Values

- NaN represents missing data

```
s = pd.Series([10, None, 30, float('nan')],
              index=['a', 'b', 'c', 'd'])
```

a	10.0
b	NaN
c	30.0
d	NaN

- s.isna()

a	False
b	True
c	False
d	True

- s.notna()

a	True
b	False
c	True
d	False

- Handle missing

s.fillna(0) →

a	10.0
b	0.0
c	30.0
d	0.0

s.dropna() →

a	10.0
c	30.0

⑩ Sorting

- Sort by values

```
s = pd.Series([3,1,4,2], index=list('abcd'))
```

s.sort_values() →

b	1
d	2
a	3
c	4

- Sort by index

s.sort_index() →

a	3
b	1
c	4
d	2

⑪ Basic Statistics

- Useful summary functions

```
s = pd.Series([10, 20, 30, 40, 50])
```

s.count()	→ 5	# non-NaN count	s.min()	→ 10	# minimum
s.sum()	→ 150	# sum of values	s.max()	→ 50	# maximum
s.mean()	→ 30.0	# average	s.std()	→ 15.81	# std deviation
s.median()	→ 30.0	# middle value	s.var()	→ 250.0	# variance

☺

Quick Summary

- ✓ Create Series
- ✓ Understand Index
- ✓ Access with loc / iloc
- ✓ Filter with conditions
- ✓ Do arithmetic
- ✓ Handle missing values
- ✓ Sort values
- ✓ Get quick statistics

Series → Simple, Powerful, Essential! ☺



Pandas DataFrame ⭐

DataFrame = Two-dimensional labeled data ☺

• This is the heart of Pandas. ❤

```
import pandas as pd

# pd is the usual alias
```

What is a DataFrame?

- A DataFrame is a 2D labeled data structure with rows and columns.
- It is like a table or a spreadsheet.
- Columns can be of different data types.

DataFrame

	Name	Age	City
0	Ravi	21	Hyd
1	Neha	22	Pune
2	Arjun	23	Delhi

← Columns

↑ Rows

① Creating DataFrames (Ways to build a DataFrame)

① From dictionaries

• Keys become columns.

```
data = { 'Name': ['Ravi','Neha','Arjun'],
         'Age' : [21, 22, 23],
         'City' : ['Hyd','Pune','Delhi']}
df = pd.DataFrame(data)
```

	Name	Age	City
0	Ravi	21	Hyd
1	Neha	22	Pune
2	Arjun	23	Delhi

② From lists

• Each inner list is a row.

```
data = [['Ravi', 21, 'Hyd'],
        ['Neha', 22, 'Pune'],
        ['Arjun', 23, 'Delhi']]
df = pd.DataFrame(data,
     columns=['Name', 'Age', 'City'])
```

	Name	Age	City
0	Ravi	21	Hyd
1	Neha	22	Pune
2	Arjun	23	Delhi

③ From NumPy arrays

• Provide column names.

```
import numpy as np
arr = np.array([1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])
df = pd.DataFrame(arr,
     columns=['A', 'B', 'C'])
```

	A	B	C
0	1	2	3
1	4	5	6
2	7	8	9

④ From CSV

• Reads data from CSV file.

```
df = pd.read_csv('data.csv')
```

data.csv →

	Name	Age	City
0	Ravi	21	Hyd
1	Neha	22	Pune
2	Arjun	23	Delhi

⑤ From JSON

• Reads JSON data.

```
df = pd.read_json('data.json')
```

{..} data.json →

	Name	Age	City
0	Ravi	21	Hyd
1	Neha	22	Pune
2	Arjun	23	Delhi

⑥ From SQL

• Reads from SQL database.

```
import sqlite3
conn = sqlite3.connect('db.sqlite')
df = pd.read_sql('SELECT * FROM
                  students', conn)
```

SQL Database →

	Name	Age	City
0	Ravi	21	Hyd
1	Neha	22	Pune
2	Arjun	23	Delhi

⑦ From Excel

• Reads from Excel file.

```
df = pd.read_excel('data.xlsx',
                   sheet_name='Sheet1')
```

X →

	Name	Age	City
0	Ravi	21	Hyd
1	Neha	22	Pune
2	Arjun	23	Delhi

Note

You can also specify parameters like:

- index_col
- usecols
- nrows
- dtype, parse_dates

and more! ☺

② Inspection (Getting to know your DataFrame)

① head()

• First 5 rows (default).

```
df.head()
```

	Name	Age	City
0	Ravi	21	Hyd
1	Neha	22	Pune
2	Arjun	23	Delhi

(up to 5 rows)

② tail()

• Last 5 rows (default).

```
df.tail()
```

	Name	Age	City
n-3	...	...	...
n-2	...	...	...
n-1	...	...	...

(last 5 rows)

③ shape

• (rows, columns)

```
df.shape
(3, 3)
```

rows, columns

④ columns

• Column labels.

```
df.columns
Index(['Name', 'Age', 'City'],
      dtype='object')
```

⑤ index

• Row labels (index).

```
df.index
RangeIndex(start=0,
           stop=3, step=1)
```

⑥ dtypes

• Data types of columns.

```
df.dtypes
Name    object
Age     int64
City    object
dtype: object
```

⑦ info()

• Summary of DataFrame.

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
```

⑧ describe()

• Summary statistics (numeric colms).

```
df.describe()
```

	Age
count	3.000000
mean	22.000000
std	1.000000
min	21.000000
25%	21.500000
50%	22.000000
75%	22.500000
max	23.000000

⑨ value_counts()

• Frequency of unique values.

```
df['City'].value_counts()
Hyd      1
Pune     1
Delhi    1
Name: City, dtype: int64
```

⑩ nunique()

• Number of unique values.

```
df['City'].nunique()
3
```

Tip

These functions help you quickly explore and understand your data! ☺

Quick Summary ⭐

- ✓ DataFrame is a 2D labeled data structure.
- ✓ You can create it from many sources.
- ✓ Inspection methods help you explore and understand your data quickly.

Data Sources (Files, DB, Web, etc.) → DataFrame (Table in Pandas) → Explore & Analyze (using Inspection methods)



Know your data better, work smarter! ☺



Pandas Selection ⭐

Select the data you need. Work like a Pro! ❤

Remember:
Rows = Axis 0
Columns = Axis 1

Our Sample DataFrame (df)

	Name	Age	City	Score
0	Ravi	21	Hyd	88
1	Neha	22	Pune	92
2	Arjun	23	Delhi	75
3	Meera	21	Hyd	90

Shape: (4, 4) Rows: 4 Cols: 4

What is Selection?

- Selection means picking specific rows and/or columns from a DataFrame.
- Pandas gives many powerful ways to do this.

Quick Decision Guide

Use This	When?
loc	When you have LABELS (row/column names)
iloc	When you have POSITIONS (integers)

① df["column"]

- Selects a single column.
- Returns a Series.

Example:

```
df["Name"]
```

0	Ravi
1	Neha
2	Arjun
3	Meera

← Series

② df[["col1", "col2"]]

- Selects multiple columns.
- Returns a DataFrame.

Example:

```
df[["Name", "Age"]]
```

	Name	Age
0	Ravi	21
1	Neha	22
2	Arjun	23
3	Meera	21

③ df.loc[]

- Label-based selection.
- Use row/column names.

Example:

```
df.loc[1:2, ["Name", "City"]]
```

	Name	City
1	Neha	Pune
2	Arjun	Delhi

Includes both start & end labels

④ df.iloc[]

- Integer position-based.
- Use row/column numbers.

Example:

```
df.iloc[0:3, 1:3]
```

	Age	City
0	21	Hyd
1	22	Pune
2	23	Delhi

End index is exclusive

② Advanced Selection Techniques

① Boolean Filtering

- Filter rows using a condition that returns True/False.

Example:

```
df[df["Age"] > 21]
```

	Name	Age	City	Score
1	Neha	22	Pune	92
2	Arjun	23	Delhi	75

② Multiple Conditions

- Combine conditions using & (and), | (or), ~ (not).

Example:

```
df[(df["Age"] > 21) &
   (df["City"] == "Hyd")]
```

Name	Age	City	Score
Meera	21	Hyd	90

③ isin( )

- Select rows where values are in a list.

Example:

```
df[df["City"].isin(["Hyd", "Pune"])]
```

	Name	Age	City	Score
0	Ravi	21	Hyd	88
1	Neha	22	Pune	92
3	Meera	21	Hyd	90

④ between( )

- Select rows within a range (inclusive).

Example:

```
df[df["Age"].between(21, 22)]
```

	Name	Age	City	Score
0	Ravi	21	Hyd	88
1	Neha	22	Pune	92
3	Meera	21	Hyd	90

⑤ query( )

- Filter using a query string.

Example:

```
df.query("Age > 21 and City == 'Pune'")
```

	Name	Age	City	Score
1	Neha	22	Pune	92

⑥ at

- Access a single value using row label & column label.

Example:

```
df.at["Arjun", "Score"]
```

→ 75

⑦ iat

- Access a single value using row position & column position.

Example:

```
df.iat[2, 3]
```

→ 75

Tiny Tips

- loc → labels (names)
- iloc → positions (numbers)
- Use .copy( ) to avoid SettingWithCopyWarning ☺

③ loc vs iloc – When to Use?

Use loc when:

- ✓ You know the row/column NAMES.
- ✓ Working with dates, strings, or categories.
- ✓ You want inclusive slicing.
- ✓ Code is more readable.

loc = Labels

VS

Use iloc when:

- ✓ You know the row/column POSITIONS.
- ✓ Doing numeric indexing or loops.
- ✓ Slightly faster (position based).
- ✓ End index is exclusive.

iloc = Integers



Practice daily. You'll master selection instinctively! ❤

⭐ **Golden Rule:** Start with simple selections → Build confidence → Combine techniques → Think like a Pandas Ninja! ☺



Pandas Data Cleaning ⭐

This is one of the most important job skills. ❤️

Clean Data
Better Analysis
Better Decisions
☺

Our Sample DataFrame (df)

	Name	Age	City	Salary	Join Date
0	Ravi	25	Hyderadad	50000	2022-01-10
1	Neha	NaN	Pune	60000	2022-02-15
2	Arjun	30	Delhi	NaN	2022-03-20
3	Meera	NaN	Hyderadad	55000	NaT
4	Karan	28	NaN	70000	2022-05-05

Shape: (5, 5)

Why Clean Data?

- Missing or wrong data can lead to wrong analysis.
- Clean data improves model accuracy & reliability.
- Saves time and builds trust in your work.

Quick Guide

Task	Use
Check missing	isna(), notna()
Remove missing	dropna()
Fill missing	fillna() / ffill / bfill
Remove dups	drop_duplicates()
Change type	astype(), convert_dtypes()
Work with text	df["col"].str

1 Missing Data

① isna()

- Detects missing values.

Example: df.isna()

	Name	Age	City	Salary	Join Date
0	False	False	False	False	False
1	False	True	False	False	False
2	False	False	False	True	False
3	False	True	False	False	True
4	False	False	True	False	False

② notna()

- Detects non-missing values.

Example: df.notna()

	Name	Age	City	Salary	Join Date
0	True	True	True	True	True
1	True	False	True	True	True
2	True	True	True	False	True
3	True	False	True	True	False
4	True	True	False	True	True

③ dropna()

- Remove rows/columns with missing values.

Example: df.dropna()

	Name	Age	City	Salary	Join Date
0	Ravi	25	Hyderadad	50000	2022-01-10
4	Karan	28	NaN	70000	2022-05-05

Default: how='any', axis=0 (rows)

④ fillna(value)

- Fill missing with a specific value.

Example: df.fillna(0)

	Name	Age	City	Salary	Join Date
0	Ravi	25	Hyderadad	50000	2022-01-10
1	Neha	0	Pune	60000	2022-02-15
2	Arjun	30	Delhi	0	2022-03-20
3	Meera	0	Hyderadad	55000	0
4	Karan	28	0	70000	2022-05-05

⑤ Forward fill (ffill)

- Fill using last valid observation.

Example: df.ffill()

	Name	Age	City	Salary	Join Date
0	Ravi	25	Hyderadad	50000	2022-01-10
1	Neha	25	Pune	60000	2022-02-15
2	Arjun	30	Delhi	60000	2022-03-20
3	Meera	30	Hyderadad	55000	2022-03-20
4	Karan	28	Hyderadad	70000	2022-05-05

⑥ Backward fill (bfill)

- Fill using next valid observation.

Example: df.bfill()

	Name	Age	City	Salary	Join Date
0	Ravi	25	Hyderadad	50000	2022-01-10
1	Neha	30	Pune	60000	6022-02-15
2	Arjun	30	Delhi	55000	2022-03-20
3	Meera	28	Hyderadad	55000	2022-05-05
4	Karan	28	NaN	70000	2022-05-05

⑦ Interpolation

- Estimate missing numeric values.

Example: df["Age"].interpolate()

	Age
0	25.0
1	27.5
2	30.0
3	29.0
4	28.0

Works best for numeric data (linear by default). ☺

2 Duplicates

① duplicated()

- Marks duplicate rows.

Example: df.duplicated()

	Name
0	False
1	False
2	False
3	False
4	True

② drop_duplicates()

- Remove duplicate rows.

Example: df.drop_duplicates()

	Name	Age	City	Salary	Join Date
0	Ravi	25	Hyderadad	50000	2022-01-10
1	Neha	NaN	Pune	60000	2022-02-15
2	Arjun	30	Delhi	NaN	2022-03-20
3	Meera	NaN	Hyderadad	55000	NaT

Keeps first occurrence by default. ☺

3 Data Types

① astype(dtype)

- Convert one or more columns.

Example:

```
df["Age"] = df["Age"].astype('int64")
```

② convert_dtypes()

- Automatically convert to best possible dtypes.

Example: df = df.convert_dtypes()

③ Numeric Conversion

- Use pandas.to_numeric()

Example:

```
df["Score"] = pd.to_numeric(
   df["Score"], errors='coerce')
```

④ Datetime Conversion

- Use pandas.to_datetime()

Example:

```
df["Join Date"] = pd.to_datetime(
  df["Join Date"], errors='coerce')
```

4 Strings

Master: df["name"].str

	Original
0	" RAVI "
1	nehA
2	Arjun Kumar
3	meera
4	RAVI_SHARMA

① lower()

```
df["name"].str.lower()
```

ravi
neha
arjun kumar
meera
ravi_sharma

② upper()

```
df["name"].str.upper()
```

RAVI
NEHA
ARJUN KUMAR
MEERA
RAVI_SHARMA

③ strip()

```
df["name"].str.strip()
```

RAVI
nehA
Arjun Kumar
meera
Ravi_Sharma

④ replace()

```
df["name"].str.replace(
  'a', 'A', regex=False)
```

RAVI
neHA
Arjun KumAr
meerA
Ravi_ShARMA

⑤ contains()

```
df["name"].str.contains('a')
```

True
True
True
True
True

⑥ startswith('R')

```
df["name"].str.startswith('R')
```

True
False
False
False
True

⑦ endswith('a')

```
df["name"].str.endswith('a')
```

False
False
False
False
True

⑧ split()

```
df["name"].str.split()
```

['RAVI']
['neha']
['Arjun', 'Kumar']
['meera']
['Ravi_SHARMA']

⑨ extract(r'([A-Z][a-z]+)')

```
df["name"].str.extract(r'([A-Z][a-z]+)')
```

RAVI
neha
Arjun
meera
Ravi

⑩ Regular Expressions

- Use powerful regex patterns with. contains(), extract(), replace(), split() etc.

Example pattern:

r'\d+' → all digits
r'^[A-Z]' → starts with capital letter ☺

Golden Rules ⭐

- ✓ Always explore your data first.
- ✓ Handle missing & duplicates early.
- ✓ Use correct data types.
- ✓ Clean text data for consistency.
- ✓ Small clean steps → Big impact!

❤️



Pandas Sorting

Arrange your data the smart way!

Sorting helps you see the most important data first!

Our Sample DataFrame (df)

	Customer	Revenue	Orders	City
0	Arjun	12000	22	Delhi
1	Neha	25000	35	Pune
2	Ravi	18000	28	Hyderabad
3	Meera	32000	40	Bangalore
4	Kiran	15000	18	Chennai
5	Pooja	28000	31	Mumbai

Shape: (6, 4) Rows: 6 Columns: 4

Why Sort Data?

- Find top / bottom performers quickly.
- Make data easy to read and analyze.
- Identify trends and patterns.
- Prepare data for reports and dashboards.

Quick Guide

Function	Use
sort_values()	Sort by column values
sort_index()	Sort by index labels
ascending	True → Ascending (small→large) False → Descending (large→small)
nlargest()	Get top N largest values
nsmallest()	Get top N smallest values

1 sort_values()

- Sorts rows by column value.
- Default order: ascending (small → large).

Example:

```
df.sort_values('Revenue')
```

	Customer	Revenue	Orders	City
0	Kiran	15000	18	Chennai
1	Ravi	18000	28	Hyderabad
2	Arjun	12000	22	Delhi
3	Pooja	28000	31	Mumbai
4	Neha	25000	35	Pune
5	Meera	32000	40	Bangalore

2 sort_index()

- Sorts the DataFrame by index labels.
- Useful when index is shuffled.

Example:

```
df.sort_index(ascending=True)
```

	Customer	Revenue	Orders	City
0	Arjun	12000	22	Delhi
1	Neha	25000	35	Pune
2	Ravi	18000	28	Hyderabad
3	Meera	32000	40	Bangalore
4	Kiran	15000	18	Chennai
5	Pooja	28000	31	Mumbai

Default: ascending=True

3 Multiple-column sorting

- Sort by more than one column.
- Uses list of column names.

Example:

```
df.sort_values(['City', 'Revenue'])
```

	Customer	Revenue	Orders	City
0	Meera	32000	40	Bangalore
1	Kiran	15000	18	Chennai
2	Arjun	12000	22	Delhi
3	Ravi	18000	28	Hyderabad
4	Pooja	28000	31	Mumbai
5	Neha	25000	35	Pune

4 Ascending / Descending

- Control sort order with ascending.
- ascending=True → small to large.
- ascending=False → large to small.

Example (Descending by Revenue):

```
df.sort_values('Revenue', ascending=False)
```

	Customer	Revenue	Orders	City
0	Meera	32000	40	Bangalore
1	Pooja	28000	31	Mumbai
2	Neha	25000	35	Pune
3	Ravi	18000	28	Hyderabad
4	Kiran	15000	18	Chennai
5	Arjun	12000	22	Delhi

5 nlargest()

- Returns top N large values.
- Faster than full sort for big data.

Example:

```
df.nlargest(3, 'Revenue')
```

	Customer	Revenue	Orders	City
0	Meera	32000	40	Bangalore
1	Pooja	28000	31	Mumbai
2	Neha	25000	35	Pune

Tip: Use nlargest() when you only need the top N rows. It's faster than sort_values().

6 nsmallest()

- Returns top N small values.
- Useful to find lowest performers.

Example:

```
df.nsmallest(3, 'Revenue')
```

	Customer	Revenue	Orders	City
0	Arjun	12000	22	Delhi
1	Kiran	15000	18	Chennai
2	Ravi	18000	28	Hyderabad

Practice

Find the top 10 customers by revenue.

```
df.nlargest(10, 'Revenue')
```

- ✓ Try multiple-column sorting for better insights.
- ✓ Sorting makes analysis faster and clearer!



Group Aggregate Gain Insights !

Pandas GroupBy

Split → Apply → Combine → Powerful Insights

Same Data New Perspective That's the power of GroupBy!

What is GroupBy?

- Groups data based on one or more columns.
- Lets you apply a function to each group.
- Very useful for aggregation, transformation and filtering.

The Split-Apply-Combine Process

1. **Split** — Split data into groups
2. **Apply** — Apply function to each group
3. **Combine** — Combine results into new object

Quick Syntax

```
df.groupby(by, as_index=True, sort=True, dropna=True)
```

- `by`: column name(s) to group by
- `as_index`: return group keys as index
- `sort`: sort group keys
- `dropna`: drop NA groups (default True)

Our Sample DataFrame

OrderID	Region	Category	Product	Revenue
1	North	Electronics	Laptop	1200
2	North	Clothing	Shirt	200
3	South	Electronics	Phone	800
4	South	Clothing	Jeans	300
5	East	Electronics	Laptop	1000
6	East	Home	Chair	400
7	West	Clothing	Shirt	250
8	West	Home	Table	700

Shape: (8, 5) Columns: 5 Rows: 8

1 Grouping One Column

- Group by a single column.
- Returns a DataFrameGroupBy object.

Example:

```
df.groupby('Region')['Revenue'].sum()
```

Region	Revenue
East	1400
North	1400
South	1100
West	950

Group wise revenue

2 Grouping Multiple Columns

- Group by two or more columns.
- Gives more detailed insights.

Example:

```
df.groupby(['Region', 'Category'])['Revenue'].sum()
```

Region	Category	Revenue
East	Electronics	1000
East	Home	400
North	Clothing	200
North	Electronics	1200
South	Clothing	300
South	Electronics	800
West	Clothing	250
West	Home	700

Multi-level insights!

3 Common Aggregation Functions

Function	Meaning	Example
sum()	Sum of values	g['Revenue'].sum()
mean()	Average value	g['Revenue'].mean()
median()	Middle value	g['Revenue'].median()
min()	Minimum value	g['Revenue'].min()
max()	Maximum value	g['Revenue'].max()
count()	Non-null count	g['Revenue'].count()
size()	Total rows (incl. NA)	g.size()
nunique()	Unique values count	g['Product'].nunique()
std()	Standard deviation	g['Revenue'].std()

Use these functions to quickly summarize your data!

4 agg() - Multiple Aggregations

- Pass a list or dict of aggregations.

Example:

```
df.groupby('Region').agg({
    'Revenue': ['sum', 'mean', 'count'],
    'OrderID': 'nunique'
})
```

	Revenue			OrderID
	sum	mean	count	nunique
East	1400	700.0	2	2
North	1400	700.0	2	2
South	1100	550.0	2	2
West	950	475.0	2	2

5 Named Aggregation

- Cleaner column names in output.

Example:

```
df.groupby('Region').agg(
    total_revenue=('Revenue', 'sum'),
    avg_revenue=('Revenue', 'mean'),
    orders=('OrderID', 'count')
)
```

Region	total_revenue	avg_revenue	orders
East	1400	700.0	2
North	1400	700.0	2
South	1100	550.0	2
West	950	475.0	2

6 transform()

- Returns result with same shape as original DataFrame.
- Useful for group-wise calculations.

Example:

```
df['Region_Total'] = df.groupby('Region')
    ['Revenue'].transform('sum')
```

OrderID	Region	Revenue	Region_Total
1	North	1200	1400
2	North	200	1400
3	South	800	1100
4	South	300	1100
...	...	...	...

7 filter()

- Keep groups that satisfy a condition.

Example:

```
df.groupby('Region').filter(
    lambda g: g['Revenue'].sum() > 1000
)
```

OrderID	Region	Category	Revenue
1	North	Electronics	1200
2	North	Clothing	200
5	East	Electronics	1000
6	East	Home	400

Only groups with total revenue > 1000

8 apply()

- Apply custom function to each group.
- Very flexible (but can be slower).

Example: Add range of revenue (max - min) for each region

```
def revenue_range(g):
    return pd.Series({
        'max': g['Revenue'].max(),
        'min': g['Revenue'].min(),
        'range': g['Revenue'].max()-g['Revenue'].min()
    })
df.groupby('Region').apply(revenue_range)
```

Region	max	min	range
East	1000	400	600
North	1200	200	1000
South	800	300	500
West	700	250	450

9 Real-World Problem

Calculate revenue by region and product category.

Solution:

```
df.groupby(['Region', 'Category'])
['Revenue'].sum().reset_index()
```

Region	Category	Revenue
East	Electronics	1000
East	Home	400
North	Clothing	200
North	Electronics	1200
South	Clothing	300
South	Electronics	800
West	Clothing	250
West	Home	700

You can now solve this without searching the internet!

Golden Rules

- ✓ Think: Split → Apply → Combine
- ✓ Use named aggregation for clean output
- ✓ Use transform() for same-shaped results
- ✓ Use filter() to keep meaningful groups
- ✓ Use apply() for custom logic
- ✓ Combine multiple aggregations with agg()
- ✓ GroupBy turns raw data into real insights

Happy Data Analysis!



Pandas Merge & Join

Combine Data ♥ Create More Possibilities

Different Tables
Same Opportunities
That's the Power
of Merge & Join!

Why Merge & Join?

- Real-world data is scattered across multiple tables.
- Helps combine related data using common keys.
- Essential for analysis, reporting and building datasets.

Think of it as: Bringing puzzle pieces together!

Our Example DataFrames

df_customers

cust_id	name	city
1	Alice	Delhi
2	Bob	Mumbai
3	Charlie	Pune
4	David	Chennai

df_orders

order_id	cust_id	product	amount
101	1	Laptop	50000
102	2	Phone	20000
103	2	Headphones	5000
104	5	Tablet	15000

Note: Customer 5 is not in customer table

merge() Syntax

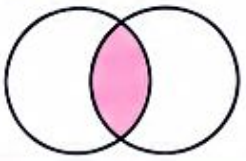
```
pd.merge(left, right,
         how='inner',
         on=None,
         left_on=None,
         right_on=None,
         suffixes=('_x', '_y'))
```

- left, right : DataFrames to merge
- how : Type of join
- on : Common column(s)
- suffixes : Suffix for overlapping cols

Types of Joins

1 Inner Join

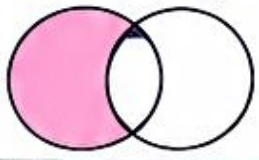
- Only matching rows from both tables.



cust_id	name	amount
1	Alice	50000
2	Bob	20000
2	Bob	5000

2 Left Join

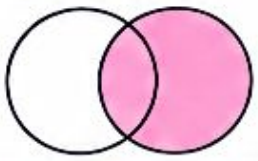
- All rows from left, matched from right.



cust_id	name	amount
1	Alice	50000
2	Bob	20000
2	Bob	5000
3	Charlie	NaN
4	David	NaN

3 Right Join

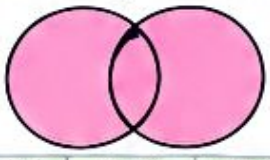
- All rows from right, matched from left.



cust_id	name	amount
1	Alice	50000
2	Bob	20000
2	Bob	5000
5	NaN	15000

4 Outer Join

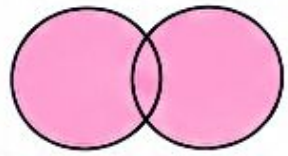
- All rows from both, fill missing with NaN.



cust_id	name	amount
1	Alice	50000
2	Bob	20000
2	Bob	5000
3	Charlie	NaN
4	David	NaN
5	NaN	15000

5 Cross Join

- Cartesian product (all combinations).



cust_id	order_id
1	101
1	102
1	103
1	104

... (total 16 rows)

Join Keys

- Column(s) used to match rows.
- Can be same name or different names (use left_on, right_on).
- Can also use multiple keys.

```
pd.merge(df1, df2,
         left_on='id1',
         right_on='id2')
```

Multiple Keys

- Merge on more than one column.

Example:

```
pd.merge(df1, df2, on=['id', 'date'])
```

id	date	value
1	2024-01-01	100
1	2024-01-02	200
2	2024-01-01	300

join() Method

- Useful when joining on index.
- Simpler syntax.

Example:

```
df1.join(df2, how='inner')
```

Equivalent to:

```
df1.merge(df2, left_index=True,
          right_index=True,
          how='inner')
```

concat() - Combining Data

- Used to concatenate DataFrames.
- Two types:
  - Vertical (stack rows)
  - Horizontal (add columns)

Vertical Concatenation

```
pd.concat([df1, df2], axis=0)
```

→ Stacks rows (one below the other)

Horizontal Concatenation

```
pd.concat([df1, df2], axis=1)
```

→ Adds columns side by side

Pandas vs SQL Equivalent

Pandas	SQL	Purpose
merge()	JOIN	Combine tables
groupby()	GROUP BY	Group and aggregate
query()	WHERE	Filter rows
agg()	Aggregation	Aggregate functions
sort_values()	ORDER BY	Sort data

Same Concept Different Syntax
Same Power!

Real-World Problem

Calculate revenue by region and product category (using multiple tables).

1. Merge orders with products (to get category)
2. Merge with customers (to get region)
3. Group by region & category and sum revenue

orders

order_id	cust_id	prod_id	amount
101	1	P1	50000
102	2	P2	20000
103	3	P1	30000

products

prod_id	category
P1	Electronics
P2	Mobile
P3	Accessories

customers

cust_id	region
1	North
2	West
3	South

Solution:

```
df = orders.merge(products, on='prod_id')
df = df.merge(customers, on='cust_id')
result = df.groupby(['region', 'category'])['amount'].sum().reset_index()
```

Final Output

region	category	revenue
North	Electronics	50000
South	Electronics	30000
West	Mobile	20000

Golden Rules

- ✓ Choose the right join type based on your need.
- ✓ Use meaningful join keys (can be multiple).
- ✓ Use suffixes to handle overlapping column names.
- ✓ Use join() for index-based merging.
- ✓ Use concat() to combine similar DataFrames.
- ✓ Pandas merge & join = SQL JOIN power!

Better Data
Happier Impact!



Same Data New Possibilities !

Pandas Reshaping

Wide ↔ Long ♥ Reshape ♥ Analyze ♥ Discover

Reshape your data
Reshape your Insights!

What is Reshaping ?

- Reshaping changes the layout of data without changing the actual information.
- Helps in better analysis, visualization and modeling.
- Two main forms: Wide & Long

Wide vs Long Format

Wide Format
(each variable in a column)

id	math	science	english
1	80	90	85
2	70	75	80
3	90	95	88

Same Information Different Shape

Long Format
(each observation in a row)

id	subject	score
1	math	80
1	science	90
1	english	85
...	...	...

When to use ?

- ✓ Use pivot()/pivot_table() for Long → Wide
- ✓ Use melt() for Wide → Long
- ✓ Use stack()/unstack() to reshape hierarchical index
- ✓ Use wide_to_long() for systematic wide → long
- ✓ Use explode() to expand list-like data

1 pivot()

- Reshape data (Long → Wide).
- Each index/column pair must be unique.

```
df.pivot(index='Student',
         columns='Subject',
         values='Score')
```

Long Format

Student	Subject	Score
A	Math	80
A	Science	90
B	Math	70
B	Science	75

→ Wide Format

Student	Math	Science
A	80	90
B	70	75

Simple & Fast when no duplicates!

2 pivot_table()

- More flexible than pivot().
- Handles duplicate entries using aggregation.
- Supports multiple agg functions.

```
df.pivot_table(index='Student',
               columns='Subject',
               values='Score', aggfunc='mean')
```

Long Format

Student	Subject	Score
A	Math	80
A	Math	100
A	Science	90
B	Math	70
B	Science	75

→ Wide Format (mean)

Student	Math	Science
A	90	90
B	70	75

Handles Duplicates Gracefully

3 melt()

- Convert from Wide → Long.
- Unpivot columns into key-value pairs.

```
pd.melt(df, id_vars=['id'],
        value_vars=['math','science'],
        var_name='subject', value_name='score')
```

Wide Format

id	math	science
1	80	90
2	70	75

→ Long Format

id	subject	score
1	math	80
1	science	90
2	Math	70
2	science	75

Useful for plotting and ML models!

4 stack()

- Move column level to row level (in MultiIndex columns).

```
df.stack()
```

Wide (MultiIndex Columns)

Subject	Math		Science	
Type	Theory	Practical	Theory	Practical
A	80	85	90	95
B	70	75	80	85

→ Stacked (Longer)

Subject	Type	Score
Math	Theory	80
	Practical	85
Science	Theory	90
	Practical	95
...	...	...

5 unstack()

- Opposite of stack().
- Move row level to column level.

```
df.unstack()
```

Stacked (MultiIndex Index)

	Subject	Score
A	Math	80
	Science	90
B	Math	70
	Science	75

→ Unstacked (Wide)

Subject	Math	Science
A	80	90
B	70	75

stack() ↔ unstack() are perfect opposites!

6 wide_to_long()

- Convert wide → long for systematic column names.

```
pd.wide_to_long(df, stubnames='score',
    i='id', j='year', sep='_', suffix='\d+')
```

Wide Format

id	score_2020	score_2021
1	80	85
2	70	75
3	90	95

→ Long Format

id	year	score
1	2020	80
1	2021	85
2	2020	70
2	2021	75
...	...	...

7 explode()

- Expand list-like elements into multiple rows.

```
df.explode('Hobbies')
```

Original

Name	Hobbies
A	[Reading, Music]
B	[Sports]
C	[Music, Travel, Food]

→ After explode()

Name	Hobbies
A	Reading
A	Music
B	Sports
C	Music
C	Travel
C	Food

Turn lists into rows!

Wide → Long → Wide: Complete Flow

Wide

id	math	science
1	80	90
2	70	75

melt() →

Long

id	subject	score
1	math	80
1	science	90
2	math	70
2	science	75

pivot() →

Back to Wide

id	math	science
1	80	90
2	70	75

Same data, different shapes, greater flexibility!

Reshaping Makes Your Data Work Harder For You!

Pandas Reshaping Golden Rules

- ✓ Use pivot() for simple Long → Wide.
- ✓ Use pivot_table() for aggregation.
- ✓ Use melt() for Wide → Long.
- ✓ Use stack()/unstack() for MultiIndex reshaping.
- ✓ Use wide_to_long() for systematic column names.
- ✓ Use explode() for list-like columns.
- ✓ Same data, new perspectives, better insights!

Happy Data Science Journey!



Small Functions Big Impact!

Pandas Apply & Functions

Apply Logic ♥ Transform Data ♥ Create Possibilities

Don't just apply() Think Smart, Use Vectorized Operations!

What are we learning ?

- map()
- apply()
- applymap() / alternatives
- transform()
- Lambda functions
- Custom functions
- When to use each
- Why vectorized ops are better

Our Sample DataFrame

id	name	age	salary	dept
1	Alice	25	50000	IT
2	Bob	30	60000	HR
3	Charlie	28	55000	IT
4	David	35	70000	Finance

Shape: (4, 5)

A Golden Rule!

Don't use apply() automatically!

- Vectorized Pandas/NumPy operations are:
  - ✓ Faster
  - ✓ More readable
  - ✓ More memory efficient
  - ✓ Often just one line of code

Quick Comparison

Method	Works On
map()	Series (element-wise)
apply()	Series / DataFrame (flexible)
applymap()	DataFrame (element-wise)
transform()	GroupBy / Series (same shape)

1 map() : Element-wise on Series

- Used for simple value mapping.
- Often used with dictionaries.

Example: Map department codes to full names

```
dept_map = {'IT': 'Information Tech',
            'HR': 'Human Resources',
            'Finance': 'Finance'}
df['dept_full'] = df['dept'].map(dept_map)
```

name	dept	dept_full
Alice	IT	Information Tech
Bob	HR	Human Resources
Charlie	IT	Information Tech
David	Finance	Finance

Simple and Powerful!

2 apply() : Flexible Function Application

- Works on Series or DataFrame.
- Can apply custom functions (row/column wise).

Example 1: Apply on a single column

```
df['age_group'] = df['age'].apply(lambda x:
    'Young' if x < 30 else 'Senior')
```

name	age	age_group
Alice	25	Young
Bob	30	Senior
Charlie	28	Young
David	35	Senior

Example 2: Apply on each row (axis=1)

```
df['bonus'] = df.apply(lambda row:
  row['salary'] * 0.1 if row['dept']=='IT' else 0,
  axis=1)
```

3 applymap() : Element-wise on DataFrame

- Apply a function to every element in a DataFrame.
- In newer Pandas versions, use DataFrame.map() (instead of applymap).

Example: Convert all values to string

```
df2 = df.applymap(str)  # older syntax
df2 = df.map(str)       # newer alternative
```

Original

id	age
1	25
2	30

→ After applymap()/map()

id	age
'1'	'25'
'2'	'30'

Good for data cleaning!

4 transform() : Same Shape Output

- Returns result with same shape as original.
- Very useful with groupby operations.

Example: Standardize salary within each dept

```
df['salary_std'] = df.groupby('dept')['salary']
  transform(lambda x: (x - x.mean()) / x.std())
```

name	dept	salary	salary_std
Alice	IT	50000	-1.00
Bob	HR	60000	NaN
Charlie	IT	55000	1.00
David	Finance	70000	NaN

Same length, same index!

5 Lambda Functions

- Anonymous small functions.
- Perfect for short, one-time logic.
- Syntax: lambda arguments: expression

Examples:

- Square a number → lambda x: x**2
- Conditional → lambda x: 'Yes' if x>0 else 'No'
- Use in apply() → df['age'].apply(lambda x: x+1)

When to use Lambda?

- ✓ Short & simple logic
- ✓ Avoids writing a full function
- ✓ Great with map(), apply(), transform()

6 Custom Functions

- For complex or reusable logic.
- Define a function and use with apply().

Example: Give performance category

```
def performance(salary):
    if salary >= 60000:
        return 'High'
    elif salary >= 50000:
        return 'Medium'
    else:
        return 'Low'
df['performance'] = df['salary'].apply(performance)
```

Reusable Code, Greater Possibilities!

Vectorized vs Apply : A Real Example

Using apply()

```
df['double'] = df['salary'].
apply(lambda x: x*2)
```

Slower (especially on large data)

Using Vectorized Operation

```
df['double'] = df['salary'] * 2
```

Much Faster & Cleaner

More Examples of Vectorized Operations

Task	Vectorized Pandas/NumPy	Avoid using apply()
Add 10 to column	df['age'] + 10	df['age'].apply(lambda x: x+10)
Uppercase strings	df['name'].str.upper()	df['name'].apply(str.upper)
Check condition	df['age'] > 30	df['age'].apply(lambda x: x>30)
Fill missing values	df['salary'].fillna(0)	df['salary'].apply(lambda x: 0 if pd.isna(x) else x)
Calculate length of strings	df['name'].str.len()	df['name'].apply(len)

Vectorized = Happy Data Science Life!

Golden Rules

- ✓ Understand the purpose of each method.
- ✓ Use vectorized operations whenever possible.
- ✓ apply() is powerful but can be slow.
- ✓ Use map() for simple value mapping.
- ✓ Use transform() when output shape should be same.
- ✓ Use custom functions for complex, reusable logic.
- ✓ Write clean, readable code.
- ✓ Small functions, big impact!

Better Tools Brighter Insights Happier You!



Dates Today, Better Tomorrow!

Pandas Datetime

Handle Time ♥ Unlock Insights ♥ Build Better Decisions

Time Data Powers Real-World Analytics!

Why Datetime is Important?

- Time data is everywhere (sales, logs, finance, IoT).
- Helps in Trend Analysis, Seasonality, Forecasting.
- Essential for real-world analytics and dashboards.

Time + Data = Insights

Common Date Formats

Format	Example
YYYY-MM-DD	2024-01-15
DD/MM/YYYY	15/01/2024
MM/DD/YYYY	01/15/2024
YYYY-MM-DD HH:MM:SS	2024-01-15 14:30:00
ISO 8601	2024-01-15T14:30:00

Our Sample DataFrams

id	order_date	amount
1	2024-01-15	250
2	2024-02-10	400
3	2024-02-28	150
4	2024-03-05	300
5	2024-03-21	500

Quick Start: pd.to_datetime()

- Converts strings to datetime.

Example:

```
df['order_date'] = pd.to_datetime(
    df['order_date'])
```

- Handles many formats automatically.
- Use format= for custom format.
- errors='coerce' sets invalid to NaT.

1 Datetime Columns & .dt Accessor

- Datetime column has dtype: datetime64[ns]
- Use .dt to access datetime properties.

Example:

```
df['order_date'] = pd.to_datetime(df['order_date'])
df['order_date'].dt.year
```

.dt gives powerful access!

order_date	year	month	day
2024-01-15	2024	1	15
2024-02-10	2024	2	10
2024-02-28	2024	2	28

2 Extracting Date Components

- Get year, month, day, week, quarter, day of week, hour etc.

Component	Code	Example (for 2024-01-15)
Year	.dt.year	2024
Month	.dt.month	1
Day	.dt.day	15
Week (ISO)	.dt.isocalendar().week	3
Quarter	.dt.quarter	1
Day of Week	.dt.dayofweek	0 (Mon=0)
Day Name	.dt.day_name()	Monday
Hour	.dt.hour	14
Minute	.dt.minute	30
Second	.dt.second	0

3 Day of Week & Week

- .dt.dayofweek → 0 (Mon) to 6 (Sun)
- .dt.day_name() → actual name
- .dt.isocalendar().week → ISO week

Example:

```
df['dow'] = df['order_date'].dt.day_name()
```

order_date	dayofweek	day_name	week
2024-01-15	0	Monday	3
2024-02-10	5	Saturday	6
2024-02-28	2	Wednesday	9

Useful for weekly analysis!

4 Timedelta & Time Differences

- Timedelta represents a duration between two dates.
- Subtract datetime columns to get time difference.

Example:

```
df['delivery_date'] = pd.to_datetime(
  ['2024-01-20', '2024-02-15', '2024-03-01'])
df['days'] = df['delivery_date'] - df['order_date']
```

order_date	delivery_date	days
2024-01-15	2024-01-20	5 days
2024-02-10	2024-02-15	5 days
2024-02-28	2024-03-01	2 days

Result is Timedelta type!

5 Timedelta Operations

- Timedelta has useful attributes.

Attribute	Meaning	Example
.days	Total days	td.days
.seconds	Seconds (within day)	td.seconds
.total_seconds()	Total seconds	td.total_seconds()

Example:

```
diff = df['delivery_date'] - df['order_date']
diff.dt.days
diff.dt.total_seconds()
```

Useful for SLA / TAT analysis!

6 Working with Time Zones

- Localize naive timestamps.
- Convert between time zones.
- Useful for global applications.

Example:

```
# Make timezone aware (UTC)
df['order_utc'] = pd.to_datetime(df['order_date']).
                  dt.tz_localize('UTC')
# Convert to another timezone
df['order_india'] = df['order_utc'].dt.tz_conver
  ('Asia/Kolkata')
```

UTC Time	India Time (IST)
2024-01-15 09:00 UTC	2024-01-15 14:30 IST
2024-02-10 12:00 UTC	2024-02-10 17:30 IST

Time zones avoid confusion!

7 More Useful Datetime Functions

- Start/End of period, formatting, now()

Function	Description	Example
pd.Timestamp.now()	Current timestamp	pd.Timestamp.now()
.dt.date	Date (no time)	df['dt'].dt.date
.dt.tz_localize()	Add timezone	.dt.tz_localize('UTC')
.dt.tz_convert()	Convert timezone	.dt.tz_convert('Asia/Kolkata')
.dt.strtfime()	Format datetime	df['dt'].dt.strffime('%Y-%m')
.dt.to_period()	Convert to period	df['dt'].dt.to_period('M')
.dt.floor('D')	Floor to day	df['dt'].dt.floor('D')
.dt.ceil('H')	Ceil to hour	df['dt'].dt.ceil('H')

Real-World Example

Analyze monthly sales, with the number of orders and average delivery time.

1. Convert dates to datetime
2. Extract month
3. Calculate delivery time
4. Group and analyze

```
df['order_date'] = pd.to_datetime(df['order_date'])
df['delivery_date'] = pd.to_datetime(df['delivery_date'])
df['month'] = df['order_date'].dt.to_period('M')
df['delivery_days'] = (df['delivery_date'] -
                       df['order_date'].dt.days
result = df.groupby('month').agg(
    total_orders=('id', 'count'),
    avg_delivery_days=('delivery_days', 'mean'),
    total_sales=('amount', 'sum')
).reset_index()
```

From Raw Dates to Real Insights!

Sample Output

month	total_orders	avg_delivery_days	total_sales
2024-01	1	5.0	250
2024-02	2	3.5	550
2024-03	2	1.0	800

Key Takeaways

- ✓ pd.to_datetime() is the first step.
- ✓ Use .dt to extract useful components.
- ✓ Timedelta helps in time difference.
- ✓ Handle time zones for global data.
- ✓ Datetime features are very powerful for real-world analytics!

Golden Rules

- ✓ Always convert to datetime early.
- ✓ Prefer vectorized .dt operations.
- ✓ Use time zones when working with global data.
- ✓ Don't use loops for datetime operations.
- ✓ Use Timedelta for time differences.
- ✓ Explore period and resample for time series.

Good Data Brighter Future Keep Learning!



Time Series Today, Better Tomorrow!

Pandas Time Series

Time Data ♥ Trends ♥ Forecasts ♥ Real Impact

Same Data Different Time Bigger Insights!

What is a Time Series?

- Data indexed by time (dates/timestamps).
- Used in finance, sales, IoT, weather, web analytics, etc.
- Order matters!

Time Series = Data + Time Index

Our Sample Data

Date	Sales
2024-01-01	120
2024-01-02	135
2024-01-03	128
2024-01-04	145
2024-01-05	160
2024-01-06	158
2024-01-07	170
...	...

Make It a Time Series

- Convert to datetime and set as index.
- Date index enables powerful time-based operations.
- Sorted index is best practice.

Example:

```
df['Date'] = pd.to_datetime(df['Date'])
df = df.set_index('Date')
df = df.sort_index()
```

Quick Syntax

Task	Syntax
Set date index	df.set_index('date')
Resample	df.resample('D').sum()
Rolling	df.rolling(window).mean()
Expanding	df.expanding().mean()
EWM	df.ewm(span=...).mean()
Shift/Lag	df.shift(1)
Diff	df.diff()
Pct Change	df.pct_change()

1 Date Indexing & Time-based Slicing

- Use datetime index for easy slicing.
- Slice by date strings.

Example:

```
df['2024-01-03':'2024-01-06']
```

Date	Sales
2024-01-03	128
2024-01-04	145
2024-01-05	160
2024-01-06	158

So Easy! No loops!

2 resample()

- Change frequency (day → week, day → month, etc).
- Supports aggregations.

Example:

```
df.resample('W').sum()
```

Week Ending	Sales
2024-01-07	1016
2024-01-14	1120
2024-01-21	980

Downsample or Upsample!

3 Rolling Windows

- Fixed-size window for moving calculations.
- Common for moving averages.

```
df['roll_7'] = df['Sales'].rolling(7).mean()
```

Date	Sales	7-Day Rolling
2024-01-01	120	NaN
2024-01-06	158	NaN
2024-01-07	170	145.14
2024-01-08	165	151.57

Smooths short-term fluctuations

4 Expanding Windows

- Window grows from the start.
- Useful for cumulative metrics.

```
df['exp_mean'] = df['Sales'].expanding().mean()
```

Date	Sales	Expanding Mean
2024-01-01	120	120.00
2024-01-02	135	127.50
2024-01-03	128	127.67
2024-01-04	145	132.00

More stable over time

5 Exponentially Weighted (EWM)

- Gives more weight to recent data.
- Useful for smoothing & volatility.

```
df['ewm_mean'] = df['Sales'].ewm(span=3).mean()
```

Date	Sales	EWM Mean
2024-01-01	120	120.00
2024-01-02	135	127.50
2024-01-03	128	127.75
2024-01-04	145	136.38

Responsive yet smooth

6 Shift, Lag and Lead

- shift(k) moves data by k periods.
- Lag = past value (shift(1))
- Lead = future value (shift(-1))

Example:

```
df['lag_1'] = df['Sales'].shift(1)
df['lead_1'] = df['Sales'].shift(-1)
```

Date	Sales	Lag(1)	Lead(1)
2024-01-03	128	135	145
2024-01-04	145	128	160
2024-01-05	160	145	158

Useful for model features!

7 diff()

- Difference from previous value.
- Good for change analysis.

```
df['diff'] = df['Sales'].diff()
```

Date	Sales	Diff
2024-01-02	135	15
2024-01-03	128	-7
2024-01-04	145	17
2024-01-05	160	15

8 Percentage Change

- Relative change from previous value.
- Common in finance and growth analysis.

```
df['pct_change'] = df['Sales'].pct_change()
```

Date	Sales	% Change
2024-01-02	135	0.1250
2024-01-03	128	-0.0519
2024-01-04	145	0.1328
2024-01-05	160	0.1034

9 Real-World Example: 7-Day Rolling Sales

Calculate 7-day rolling sales.

1. Convert to datetime & set index
2. Use rolling(7).sum() or .mean()
3. Analyze the trend

```
df['Date'] = pd.to_datetime(df['Date'])
df = df.set_index('Date')
df['rolling_7d_sales'] = df['Sales'].rolling(7).sum()
```

Date	Sales	7-Day Rolling Sales
2024-01-06	158	NaN
2024-01-07	170	1016
2024-01-08	165	1120
2024-01-09	162	1229

Total sales in last 7 days!

Key Takeaways

- ✓ Always use datetime index.
- ✓ resample() for different frequencies.
- ✓ rolling(): moving calculations.
- ✓ expanding(): growing window.
- ✓ ewm(): weighted mean.
- ✓ shift(), diff(), pct_change() for change analysis.
- ✓ Time-based slicing makes analysis super easy.

Time Series Analysis Turns Data Into Real-World Impact!

Golden Rules

- ✓ Use datetime index for all time series work.
- ✓ Prefer vectorized Pandas operations.
- ✓ Use rolling(), expanding(), ewm() as per need.
- ✓ Use shift(), diff(), pct_change() for feature engineering.
- ✓ Explore, visualize, and create impactful insights!

Happy Analysing!



Small Change Big Impact!

Pandas Categoricals

Categorical Data ♥ Faster ♥ Smaller ♥ Smarter

Use Categories
Save Memory
Improve Performance
Get Better Insights!

What are Categoricals?

- A special dtype for variables with a limited set of possible values (categories).
- Stores data as integer codes internally.
- Great for nominal or ordinal data.

Same information, less memory!

Our Sample Data

id	city	plan	rating
1	Delhi	Basic	4
2	Mumbai	Premium	5
3	Delhi	Basic	3
4	Bangalore	Premium	5
5	Chennai	Standard	4
...	...	...	...

Convert to category dtype

- Use astype('category')
- Categories are stored as codes + category list.

Example:

```
df['city'] = df['city'].astype('category')
```

Same values, smaller memory, faster operations!

Quick Syntax

Task	Syntax
Convert to category	astype('category')
Get categories	.cat.categories
Get codes	.cat.codes
Set categories	.cat.set_categories()
Reorder categories	.cat.reorder_categories()
Ordered categories	pd.Categorical(..., ordered=True)
Add/Remove cats	.cat.add_categories() .cat.remove_categories()
Rename categories	.cat.rename_categories()

1 category dtype in Action

- Memory efficient representation using integer codes.
- Same behavior as object for most operations.

Example:

Object dtype

city	dtype
Delhi	object
Mumbai	object
Bangalore	object
Chennai	object

Convert →

Category dtype

city	dtype	codes
Delhi	category	0
Mumbai	category	1
Bangalore	category	2
Chennai	category	3

Internally stored as 0,1,2,3 instead of full strings!

2 Ordered Categories

- Categories can have a meaningful order.
- Useful for ratings, education levels, low/medium/high etc.

Example:

```
levels = ['Low','Medium','High']
cat_type = pd.CategoricalDtype(
           categories=levels, ordered=True)
df['priority'] = df['priority'].astype(cat_type)
```

priority	code
Low	0
Medium	1
High	2

Now sorting follows this meaningful order!

3 Category Operations

- Access categories, codes, rename, add or remove categories.
- Useful methods with examples:

Operation	Example
Get categories	df['city'].cat.categories
Get codes	df['city'].cat.codes
Rename cat	df['city'].cat.rename_categories ({'Delhi':'DL','Mumbai':'MH'})
Add category	df['city'].cat.add_categories(['Kolkata'])
Remove category	df['city'].cat.remove_categories(['Chennai'])
Set order	df['city'].cat.reorder_categories (['Mumbai','Delhi','Bangalore'], ordered=True)

4 Memory Optimization

- Categories use much less memory than object dtype, especially with repeated values.

Example:

```
df = pd.DataFrame({'city': cities})
```

dtype	memory_usage
object	6508 bytes
category	374 bytes

~ 90% less memory!

5 When Categoricals Improve Performance

- Better memory efficiency
- Faster groupby, merge, value_counts()
- Faster sorting (especially with ordered cats)
- Ideal for low-cardinality columns (few unique values)

Best Use Cases

- ✓ Columns with repeated values
- ✓ Low number of unique categories
- ✓ Groupby/aggregation operations
- ✓ Joins/merges on categorical columns
- ✓ Machine learning features, dashboards and reporting

Less memory + Faster computations = Happier You!

6 Real-World Example

- Sales data with product category

Before (object)

product	sales
Laptop	12000
Phone	8000
Phone	6000
Laptop	10000

Memory: 6272 bytes

After (category)

product	sales
Laptop	12000
Phone	8000
Phone	6000
Laptop	10000

Memory: 360 bytes

Insights

- Same results, much less memory
- Faster groupby and filtering
- Scales better for large datasets

Extra Examples

Value counts (faster with category)

```
df['city'].value_counts()
```

city	count
Delhi	2
Mumbai	1
Bangalore	1
Chennai	1

Groupby on categorical column

```
df.groupby('plan')['sales'].sum()
```

plan	sales
Basic	18000
Premium	18000
Standard	5000

Sorting with ordered categories

```
df.sort_values('priority')
```

task	priority
Task A	Low
Task C	Medium
Task B	High

Categories Make Your Data Work Smarter!

Key Takeaways

- ✓ Use category dtype for low-cardinality data.
- ✓ Ordered categories help with meaningful sorting.
- ✓ Categoricals save memory and improve performance.
- ✓ Use .cat accessor for powerful operations.
- ✓ Great for real-world analytics and ML.

Golden Rules

- ✓ Use categoricals for columns with few unique values.
- ✓ Prefer ordered categories when there is a natural order.
- ✓ Always check memory usage for large datasets.
- ✓ Categoricals make your code faster, cleaner and more efficient!

Better Data
Brighter Decisions
Keep Learning!



Multiple Levels Multiple Possibilities!

Pandas MultiIndex

Hierarchical Data ♥ More Power ♥ Real-World Ready

Same Data Different Levels Bigger Insights!

What is MultiIndex ?

- Hierarchical indexing with multiple levels (rows or columns).
- Helps work with high-dimensional data in a structured way.
- Very useful for real-world datasets (e.g., sales by region and product).

Think: Index within an Index!

Our Sample Data (MultiIndex Rows)

Region	Product	Sales	Profit
North	Laptop	1200	240
North	Phone	800	120
South	Laptop	1000	200
South	Phone	600	90

Index names: ['Region', 'Product']

Our Sample Data (MultiIndex Columns)

	Sales		Profit	
Region	2023	2024	2023	2024
North	1000	1200	200	240
South	800	1000	150	200
East	600	700	100	120

Column levels: ['Metric', 'Year']

Quick Syntax

Task	Syntax
Create MultiIndex	pd.MultiIndex.from_...
Select data	df.loc[] / df.xs()
Cross-section	df.xs(key, level=...)
Swap levels	df.swaplevel(i, j)
Reorder levels	df.reorder_levels([...])
Stack / Unstack	df.stack() / df.unstack()
Reset index	df.reset_index()

1 Creating a MultiIndex

- Use pd.MultiIndex.from_arrays() or from_tuples().

Example (from arrays):

```
arrays = [['North', 'North', 'South', 'South'],
          ['Laptop', 'Phone', 'Laptop', 'Phone']]
index = pd.MultiIndex.from_arrays(arrays,
                names=['Region', 'Product'])
```

Create meaningful hierarchies!

2 Selecting from MultiIndex

- Use .loc with tuples or list of levels.
- You can select a single level or multiple levels.

Example:

```
df.loc['North']      # all products in North
df.loc[('North','Laptop')]   # specific
df.loc[:,'Laptop']       # all regions, Laptop
```

Easy and Powerful Selection!

3 xs() : Cross-section

- Extracts a cross-section at a particular level.
- More convenient than .loc for complex cases.

Example:

```
df.xs('North', level='Region')
```

Product	Sales	Profit
Laptop	1200	240
Phone	800	120

Great for slicing one level!

4 swaplevel() : Swap Level Order

- Swap the order of index levels.
- Useful for reorganization.

Example:

```
df.swaplevel('Region', 'Product').sort_index()
```

Original

Region	Product	Sales
North	Laptop	1200
North	Phone	800
South	Laptop	1000
South	Phone	600

→ After swaplevel()

Product	Region	Sales
Laptop	North	1200
Laptop	South	1000
Phone	North	800
Phone	South	600

5 reorder_levels() : Custom Order

- Reorder the levels to a new order.

Example:

```
df.reorder_levels(['Product', 'Region'])
```

Original Order

Region	Product	Sales
North	Laptop	1200
North	Phone	800
South	Laptop	1000
South	Phone	600

→ Reordered

Product	Region	Sales
Laptop	North	1200
Laptop	South	1000
Phone	North	800
Phone	South	600

Same data, new perspective!

6 Stack and Unstack

- stack(): moves columns to index.
- unstack(): moves index to columns.
- Very useful for reshaping data.

Example:

```
stacked = df.stack()          # columns → index
unstacked = stacked.unstack()  # index → columns
```

Original (MultiIndex Columns)

	Sales		Profit	
	2023	2024	2023	2024
North	1000	1200	200	240
South	800	1000	150	200

→ After stack()

Region	Metric	Year	Value
North	Sales	2023	1000
North	Sales	2024	1200
North	Profit	2023	200
...	...	...	...

7 Multi-level Columns

- Columns can also have multiple levels.
- Access using tuples or column level names.

Example: df[('Sales', '2024')]

	Sales		Profit	
	2023	2024	2023	2024
North	1000	1200	200	240
South	800	1000	150	200

Useful for wide data with multiple dimensions!

8 Resetting Index

- Converts index levels back to columns.
- Useful for saving or further processing.

Example:

```
df.reset_index()
```

With MultiIndex

Region	Product	Sales
North	Laptop	1200
North	Phone	800
South	Laptop	1000
South	Phone	600

→ After reset_index()

Region	Product	Sales
North	Laptop	1200
North	Phone	800
South	Laptop	1000
South	Phone	600

Real-World Example

Total Sales by Region (using MultiIndex)

1. Create MultiIndex from arrays.
2. Calculate total sales by region.
3. Show result.

Code:

```
total_sales = df.groupby(level='Region')['Sales'].sum()
```

Output:

Region	Total Sales
North	2000
South	1600

MultiIndex makes complex analysis simple!

Golden Rules

- ✓ Use MultiIndex for hierarchical data.
- ✓ xs() is great for cross-sections.
- ✓ Use stack()/unstack() for reshaping.
- ✓ Reset index when you need flat data.
- ✓ Multi-level columns are powerful for wide datasets.

More Skills Brighter Career Keep Learning!



Pandas Input/Output

Different Formats ♥ Same Powerful Pandas ♥ Real-World Impact

Data In
Insights Out
Pandas Makes
It Possible!

Why I/O is Important ?

- Data comes from many sources and formats.
- Lets you work with real-world data easily.
- Enable data sharing, storage and deployment.
- Choose the right format for your use case.

Good Data Access = Greater Possibilities!

Our Sample DataFrame

id	name	department	salary
1	Alice	IT	50000
2	Bob	HR	60000
3	Charlie	Finance	55000
4	David	Marketing	65000

```
df = pd.DataFrame(...)
```

Shape: (4, 4)

General Syntax

- Read: `pd.read_format(...)`
- Write: `df.to_format(...)`
- Many functions have useful parameters like:
  - ✓ filepath / buffer
  - ✓ index
  - ✓ encoding
  - ✓ compression
  - ✓ if_exists (for SQL)

Same Pattern, Easy to Remember!

Quick Comparison

Format	Read Function	Write Function
CSV	read_csv()	to_csv()
Excel	read_excel()	to_excel()
JSON	read_json()	to_json()
SQL	read_sql()	to_sql()
Parquet	read_parquet()	to_parquet()
Feather	read_feather()	to_feather()
Pickle	read_pickle()	to_pickle()
HTML	read_html()	to_html()
XML	read_xml()	to_xml()
Clipboard	read_clipboard()	to_clipboard()

1 CSV - Simple & Popular

- Text format, works everywhere.
- Good for small to medium data.

Read CSV

```
df = pd.read_csv('data.csv')
```

Write CSV

```
df.to_csv('output.csv',
    index=False)
```

Tip: Use index=False to avoid extra index column!



2 Excel - For Business Users

- Supports .xlsx and .xls files.
- Useful for reports and sharing.

Read Excel

```
df = pd.read_excel('data.xlsx',
    sheet_name='Sheet1')
```

Write Excel

```
df.to_excel('output.xlsx',
    index=False,
    sheet_name='Result')
```



Works with multiple sheets!

3 JSON - For Web & APIs

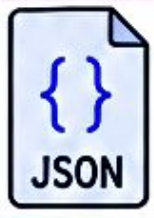
- Lightweight, human-readable.
- Great for web applications.

Read JSON

```
df = pd.read_json('data.json')
```

Write JSON

```
df.to_json('output.json',
    orient='records',
    indent=2)
```



Great for data exchange between systems!

4 SQL - Work with Databases

- Read data using SQL queries.
- Write DataFrame to a SQL table.

Read from SQL

```
df = pd.read_sql('SELECT *
    FROM employees', conn)
```

Write to SQL

```
df.to_sql('employees', conn,
    if_exists='replace',
    index=False)
```



Works with SQLite, MySQL, PostgreSQL, SQL Server, etc.

5 Other Important Formats

Pandas supports many more formats for real-world analytics.

Parquet ★ (Most Important!)

- Columnar format, high performance.
- Smaller file size.
- Great for big data & cloud analytics.

```
df.to_parquet('data.parquet')
df = pd.read_parquet('data.parquet')
```



Best choice for modern analytics!

Feather

- Very fast for small/medium data.
- Good for inter-process communication.

```
df.to_feather('data.feather')
df = pd.read_feather('data.feather')
```



Super fast!

Pickle

- Python-specific binary format.
- Preserves Python objects.
- Not secure for untrusted data.

```
df.to_pickle('data.pkl')
df = pd.read_pickle('data.pkl')
```



Good for saving intermediate results!

HTML

- Read/write HTML tables.
- Useful for web reports.

```
dfs = pd.read_html('table.html')
df.to_html('report.html',
    index=False)
```



Easy web sharing!

Clipboard

- Quickly copy/paste data.
- Useful for small datasets.

```
df = pd.read_clipboard()
df.to_clipboard()
```



Perfect for quick analysis!

XML

- Read/write XML data.
- Used in many enterprise systems.

```
df = pd.read_xml('data.xml')
df.to_xml('output.xml',
    index=False)
```



Useful for integration!

6 Common Parameters

Parameter	Use
filepath/buffer	File path or file-like object
index	Write row index or not
encoding	File encoding (e.g., 'utf-8')
compression	'gzip', 'bz2', 'zip', 'snappy'
sheet_name	Excel sheet to read/write
if_exists	For to_sql(): 'fail','replace','append'
orient	JSON format: 'records','split','table'

Real-World Example: Save & Load Analysis

1. Save cleaned data to Parquet.
2. Load it later and continue analysis.

```
# Save
df.to_parquet('cleaned_data.parquet', index=False)

# Load later
df2 = pd.read_parquet('cleaned_data.parquet')
```

Same data, smaller size, much faster!

Key Takeaways

- ✓ Choose the right format for your use case.
- ✓ Parquet is best for modern, large-scale analytics.
- ✓ CSV and Excel are great for sharing.
- ✓ JSON is ideal for web and APIs.
- ✓ SQL helps you work with databases.
- ✓ Pandas supports many other formats like Feather, Pickle, HTML, Clipboard and XML.
- ✓ Mastering I/O makes you a more versatile and job-ready data professional!

Golden Rules

- ✓ Use Parquet for big data and performance.
- ✓ Use index=False when you don't need the index.
- ✓ Always handle file paths and exceptions.
- ✓ Know the strengths and limitations of each format.

Better Data
Brighter Careers
Keep Learning!



Good Plots Brighter Insights!

Pandas Visualization

From Data to Stories ♥ Explore ♥ Understand ♥ Make Impact

Visualize Analyze Communicate Create Impact!

Why Visualize Data?

- Easier to understand patterns
- Find trends, outliers, relationships
- Communicate insights effectively
- Supports better decision making

A picture speaks louder than 1000 rows!

Basic Pandas Plot

- Pandas has a simple plotting interface using Matplotlib.
- Syntax: `df.plot(..,)`

Example:

```
df['sales'].plot(title='Sales Trend')
```

Quick & Easy!

Sample Data

date	sales	category	value
2024-01-01	120	A	10
2024-01-02	150	B	12
2024-01-03	170	A	13
2024-01-04	160	B	11
2024-01-05	180	A	15

Same Data, Many Stories!

Common df.plot() Options

- kind = 'line' | 'bar' | 'hist' | 'box' | 'scatter'
- x, y: columns to plot
- title, xlabel, ylabel
- figsize, color, legend
- grid=True, style=..,

Start simple, then go deeper!

Common Plot Types with Pandas

1 Line Plot

- Shows trend over time
- Great for time series

```
df.plot(kind='line',
    x='date', y='sales')
```

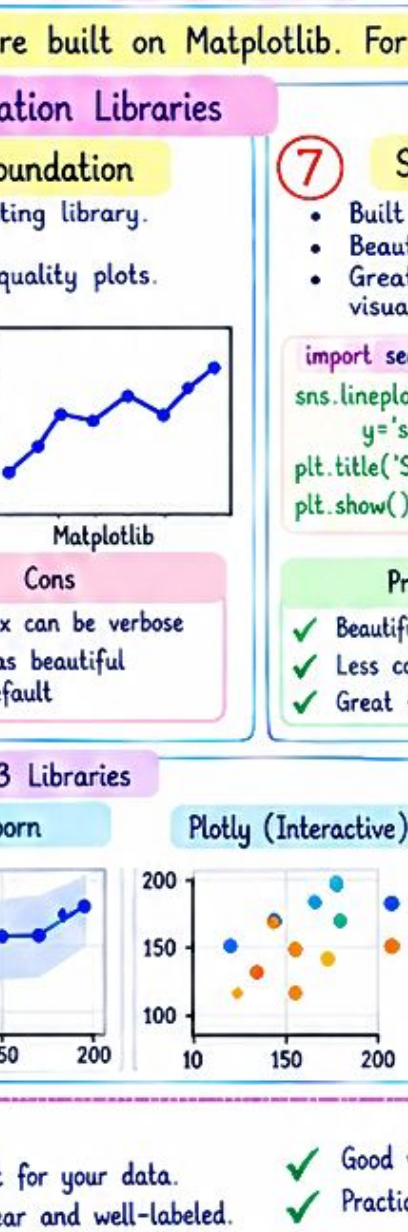


Shows continuous change

2 Bar Chart

- Compares categories
- Good for discrete data

```
df.plot(kind='bar',
    x='category', y='value')
```

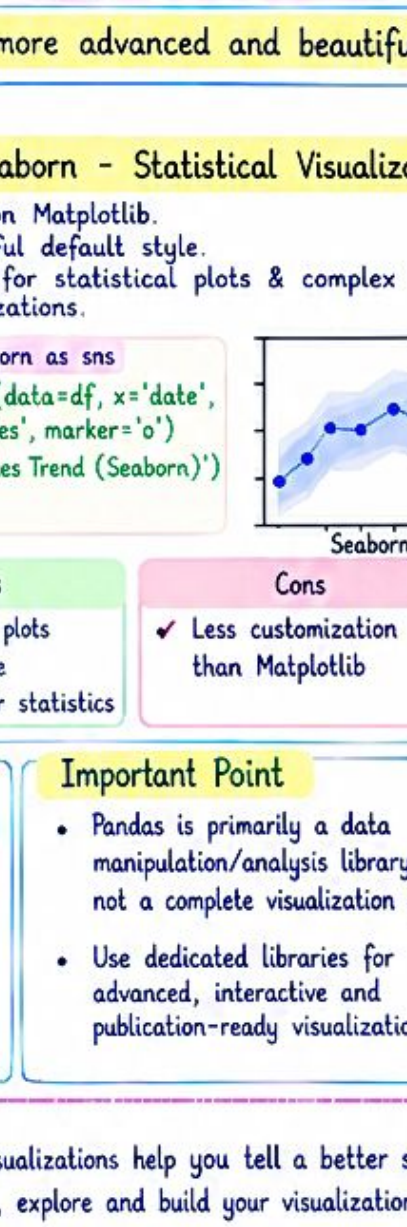


Compare different groups

3 Histogram

- Shows distribution of a numeric variable

```
df['value'].plot(kind='hist',
    bins=5, color='skyblue')
```



Understand data distribution

4 Box Plot

- Shows median, spread, outliers

```
df.boxplot(column='value')
```

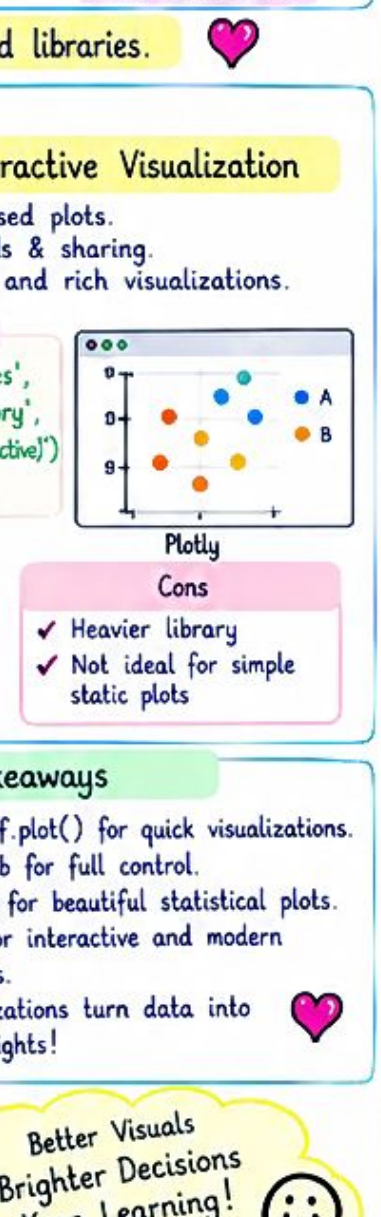


Identify outliers & spread

5 Scatter Plot

- Shows relationship
- between two variables

```
df.plot(kind='scatter',
    x='sales', y='value',
    color='purple')
```



Find correlations

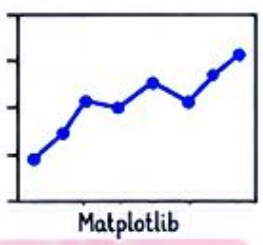
Note: Pandas plots are built on Matplotlib. For more advanced and beautiful visualizations, use dedicated libraries. ♥

Dedicated Visualization Libraries

6 Matplotlib - The Foundation

- Most popular, low-level plotting library.
- Highly customizable.
- Good for static, publication-quality plots.

```
import matplotlib.pyplot as plt
plt.plot(df['date'], df['sales'])
plt.title('Sales Trend')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.show()
```



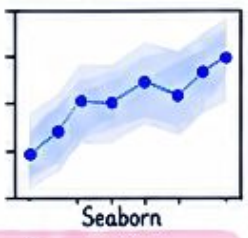
Pros
- ✓ Flexible & powerful
- ✓ Great community
- ✓ Works everywhere

Cons
- ✓ Syntax can be verbose
- ✓ Not as beautiful by default

7 Seaborn - Statistical Visualization

- Built on Matplotlib.
- Beautiful default style.
- Great for statistical plots & complex visualizations.

```
import seaborn as sns
sns.lineplot(data=df, x='date',
    y='sales', marker='o')
plt.title('Sales Trend (Seaborn)')
plt.show()
```



Pros
- ✓ Beautiful plots
- ✓ Less code
- ✓ Great for statistics

Cons
- ✓ Less customization than Matplotlib

8 Plotly - Interactive Visualization

- Interactive, web-based plots.
- Great for dashboards & sharing.
- Supports 3D, maps, and rich visualizations.

```
import plotly.express as px
fig = px.scatter(df, x='sales',
    y='value', color='category',
    title='Sales vs Value (Interactive)')
fig.show()
```

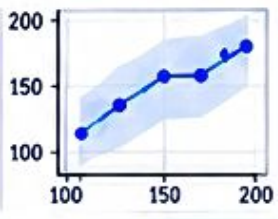

Pros
- ✓ Interactive & dynamic
- ✓ Beautiful & modern
- ✓ Great for dashboards

Cons
- ✓ Heavier library
- ✓ Not ideal for simple static plots

Visualizing the Same Data in 3 Libraries

Matplotlib

Seaborn



Plotly (Interactive)

Important Point

- Pandas is primarily a data manipulation/analysis library, not a complete visualization system.
- Use dedicated libraries for advanced, interactive and publication-ready visualizations.

Use the right tool for the right job!

Key Takeaways

- ✓ Start with df.plot() for quick visualizations.
- ✓ Use Matplotlib for full control.
- ✓ Use Seaborn for beautiful statistical plots.
- ✓ Use Plotly for interactive and modern visualizations.
- ✓ Good visualizations turn data into valuable insights! ♥

Golden Rules

- ✓ Always choose the right plot for your data.
- ✓ Keep your charts simple, clear and well-labeled.
- ✓ Good visualizations help you tell a better story.
- ✓ Practice, explore and build your visualization skills!

♥

Better Visuals Brighter Decisions Keep Learning!



Analyze Deeper Build Smarter Solutions!

Pandas Advanced Analytics

Window Functions ♥ Trends ♥ Statistics ♥ Real-World Insights

Same Data Bigger Insights Stronger Impact!

Why Advanced Analytics ?

- Find trends and patterns
- Handle time series & sequential data
- Make data-driven decisions
- Essential for real-world analytics, finance, ML

It's not just data, It's the story behind it!

Our Sample Data (Time Series)

Date	Sales	Cost	Profit
2024-01-01	100	60	40
2024-01-02	120	70	50
2024-01-03	140	80	60
2024-01-04	160	90	70
2024-01-05	180	100	80
...	...	...	...

General Syntax

- Works on Series and DataFrame
- Most functions have a window parameter
- Use .rolling(), .expanding(), .ewm() for window operations

Same Data, Multiple Possibilities!

Quick Function Reference

Task	Syntax
Rolling window	df.rolling(window)
Expanding window	df.expanding()
Exponentially weighted	df.ewm(span=...)
Shift(lag/lead)	df.shift(n)
Difference	df.diff(n)
Percentage change	df.pct_change(n)
Ranking	df.rank(method=...
Cumulative sum	df.cumsum()
Quantiles	df.quantile(q)
Correlation	df.corr()
Covariance	df.cov()

Powerful Functions, Real Impact!

1 rolling()

- Fixed-size moving window.
- Useful for moving averages.

```
df['Sales'].rolling(3).mean()
```

Date	Sales	3-MA
01-01	100	NaN
01-02	120	NaN
01-03	140	120.00
01-04	160	140.00
01-05	180	160.00

Smooths short-term noise!

2 expanding()

- Window grows from start.
- Useful for cumulative average.

```
df['Sales'].expanding().mean()
```

Date	Sales	Expanding Mean
01-01	100	100.00
01-02	120	110.00
01-03	140	120.00
01-04	160	130.00
01-05	180	140.00

Uses all past data!

3 ewm()

- Exponentially weighted moving average (more weight to recent).

```
df['Sales'].ewm(span=3).mean()
```

Date	Sales	EWM (span=3)
01-01	100	100.00
01-02	120	110.00
01-03	140	125.00
01-04	160	142.50
01-05	180	161.25

Responsive & smooth!

4 shift()

- Shifts data by periods.
- Lag = past value, Lead = future.

```
df['Sales'].shift(1)
```

Date	Sales	Lag(1)
01-01	100	NaN
01-02	120	100
01-03	140	120
01-04	160	140
01-05	180	160

Useful for lag features!

5 diff()

- Difference from previous value.
- Useful for find changes.

```
df['Sales'].diff()
```

Date	Sales	Diff
01-01	100	NaN
01-02	120	20
01-03	140	20
01-04	160	20
01-05	180	20

Highlights actual change!

6 pct_change()

- Relative change from previous.
- Useful for growth analysis.

```
df['Sales'].pct_change()
```

Date	Sales	Pct Change
01-01	100	NaN
01-02	120	0.2000
01-03	140	0.1667
01-04	160	0.1429
01-05	180	0.1250

Shows % growth!

7 Ranking

- Assign ranks to values.
- Methods: average, min, max, first, dense.

```
df['Sales'].rank(method='min')
```

Sales	Rank (min)
100	1.0
120	2.0
140	3.0
160	4.0
180	5.0

Useful for comparisons!

8 Cumulative Calculations

- Running total over time.
- Functions: cumsum(), cumprod(), cummax(), cummin()

```
df['Sales'].cumsum()
```

Date	Sales	Cumulative Sum
01-01	100	100
01-02	120	220
01-03	140	360
01-04	160	520
01-05	180	700

Track progress over time!

9 Window Functions (Generic)

- Operations over a moving/expanding window (mean, sum, min, max, std).

```
df.rolling(3).agg(['mean','sum','std'])
```

Date	Mean	Sum	Std
01-01	NaN	NaN	NaN
01-02	NaN	NaN	NaN
01-03	120	360	20.00
01-04	140	420	20.00
01-05	180	480	20.20
...	...	...	...

Powerful for feature engineering!

10 Quantiles

- Find value at given quantile.
- Useful for percentiles & outlier analysis.

```
df['Sales'].quantile([0.25,0.5,0.75])
```

Quantile	Value
0.25	120
0.50	140
0.75	160

Understand data distribution!

11 Correlation

- Measures linear relationship (-1 to 1).
- Useful for feature selection.

```
df.corr()
```

	Sales	Cost	Profit
Sales	1.00	0.99	1.00
Cost	0.99	1.00	0.99
Profit	1.00	0.99	1.00

Closer to 1 = Strong Positive Relationship

12 Covariance

- Measures joint variability.
- Positive: move together
- Negative: move opposite

```
df.cov()
```

	Sales	Cost	Profit
Sales	800	800	800
Cost	800	800	800
Profit	800	800	800

Scale dependent, used in risk analysis

Real-World Example

Date	Sales	7-Day MA	Cumulative	Pct Change
01-01	100	NaN	100	NaN
01-02	120	NaN	220	0.2000
01-03	140	NaN	360	0.1667
01-04	160	NaN	520	0.1429
01-05	180	NaN	700	0.1250
...	...	...	...	...

Key Takeaways

- ✓ rolling() for short-term trends.
- ✓ expanding() for cumulative insights.
- ✓ ewm() for weighted trends.
- ✓ shift(), diff(), pct_change() for time-based features.
- ✓ Ranking, quantiles, correlation, covariance are crucial for advanced analytics.
- ✓ Window functions power real-world analytics, finance and machine learning.

Analyze Today, Lead Tomorrow!

Golden Rules

- ✓ Always understand your data.
- ✓ Use window functions for time series.
- ✓ Check correlation before building models.
- ✓ Combine these functions for powerful feature engineering.

Better Analytics Brighter Career!

Remember: Pandas gives you the tools ♥ You create the impact! ♥ Good Data ♥ Good Decisions ♥ A Better Tomorrow ♥

Pandas Performance Optimization

Faster Pandas Happier Projects!

Same Code ♥ Less Time ♥ Less Memory ♥ Bigger Impact

Optimize Today, Scale Tomorrow!

Why Performance Matters?

- Large datasets take time and memory.
- Faster code = more productive you.
- Efficient code scales to real-world applications.

Optimization turns good code into great code!

A Quick Example: Loop vs Vectorization

Using Python loop (slow!)

```
total = 0
for x in df['sales']:
    total += x
```

15.2 s

Using Vectorization (fast!)

```
total = df['sales'].sum()
```

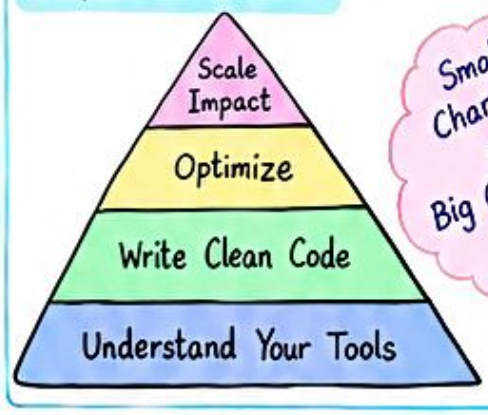
0.01 s

Same result, 1000x faster!

Golden Rule

- ✓ Prefer vectorized operations built into Pandas/NumPy.
- ✓ Avoid Python loops.
- ✓ Avoid unnecessary apply().
- ✓ Use the right data types.
- ✓ Read only what you need.
- ✓ Profile and benchmark before optimizing.

Performance Mindset



Small Changes = Big Gains!

1 Vectorization

- Use built-in Pandas/NumPy operations (C/optimized code).
- Avoid explicit loops.

Example:

```
df['tax'] = df['sales'] * 0.18
```

Vectorized operations are orders of magnitude faster!

2 Avoiding Loops

- for loops are slow in Python.
- Use vectorized operations, list/array operations, or built-in methods.

Slow ✗

```
for i in range(len(df)):
    df.loc[i, 'x'] = ...
```

Fast ✓

```
df['x'] = ...
```

3 Avoiding unnecessary apply()

- apply() is flexible but slower than vectorized methods.
- Try to use built-in methods like map(), str.*, dt.*, etc.

Slow

```
df['city'].apply(len)
```

Fast ✓

```
df['city'].str.len()
```

Use apply() only when no vectorized alternative exists!

4 Efficient Filtering

- Use boolean masks & query().
- Avoid chained filtering.

```
df[(df['sales'] > 100) &
   (df['region'] == 'North')]
```

Or using query()

```
df.query('sales > 100 and
region == "North"')
```

Clean, Fast, Readable!

5 Efficient Joins

- Use appropriate join type (inner, left, right).
- Set index for faster joins when possible.
- Avoid many-to-many joins if not needed.

```
df1.merge(df2, on='id',
how='left')
```

Smaller data + right join = big speedup!

6 Memory Optimization

- Use efficient dtypes.
- Drop unnecessary columns.
- Use inplace=True when safe.

```
df = df.drop(['col1', 'col2'],
  axis=1, inplace=True)
```

Less Memory More Data Possibilities!

7 Appropriate Dtypes

- Use smallest suitable dtype.

Data	Good dtype	Memory (approx)
Integers	int8/int16	1-2 bytes
Floats	float32	4 bytes
Strings	category	very small
Booleans	bool	1 byte
Dates	datetime64	8 bytes

Right dtype = less memory, faster computation!

8 Categorical dtype

- Use for low-cardinality string columns.
- Saves memory & improves performance (groupby, joins).

```
df['city'] = df['city'].astype('category')
```

Before	Chicions	Amount
100,000	(:fic	6.5 MB
After	100,000	0.8 MB

~ 8x less memory!

9 Chunk Processing

- Read large files in chunks.
- Process each chunk and combine results.

```
for chunk in pd.read_csv(
'bigfile.csv', chunksize=100000):
  process(chunk)
```

Handle bigger-than-memory files!

10 Read Only Required Columns

- Use the usecols parameter to read specific columns only.

```
pd.read_csv('data.csv',
 usecols=['id', 'name', 'sales'])
```

Less I/O
Faster Load
Lower Memory

11 Read with Appropriate Dtypes

- Specify dtypes while reading.

```
pd.read_csv('data.csv',
  dtype={'id': 'int32',
         'city': 'category',
         'sales': 'float32'},
  parse_dates=['date'])
```

Save memory from the start!

12 Parquet (Most Important!)

- Columnar format, compressed, faster read/write.
- Great for big data & modern analytics.

```
df.to_parquet('data.parquet')
df = pd.read_parquet('data.parquet')
```

PARQUET

- ✓ Smaller size
- ✓ Faster I/O
- ✓ Supports schema
- ✓ Industry standard

13 Profiling

- Find which parts of code are slow (time & memory).
- Use cProfile for detailed analysis.
- Use memory profiling to check memory usage.

14 Benchmarking with timeit

- Measure small code snippets accurately.

Example:

```
import timeit
timeit.timeit("df['sales'].sum()",
      globals=globals(),
      number=1000)
```

Get accurate execution time!

15 cProfile

- Profile full scripts and get detailed function-level report.

Example:

```
import cProfile
cProfile.run('your_function()')
```

Find bottlenecks and optimize!

16 Memory Profiling Concepts

- Check memory usage of objects.
- Use memory_profiler or tracemalloc.

Example:

```
from memory_profiler import profile
@profile
def your_function():
    ...
```

Know where your memory goes!

Key Takeaways

- ✓ Write vectorized code, avoid loops.
- ✓ Use proper dtypes and categorical.
- ✓ Read only what you need (columns, dtypes, chunks).
- ✓ Parquet is the best format for big data.
- ✓ Profile and benchmark before an after optimization.

Better Performance
Brighter Opportunities
Keep Learning!

♥ Optimized Pandas = Faster Insights = Greater Impact = A Stronger Career ♥

You've Got This!



Less Memory More Possibilities!

Pandas Memory Management

Understand Types ♥ Optimize Usage ♥ Handle Big Data ♥ Real-World Impact

Manage Memory Today, Enable Bigger Tomorrow!

Why Memory Management?

- Bigger datasets are common (GBs to TBs).
- Efficient memory usage improves performance.
- Avoids crashes and enables real-world applications.

Good Memory Management = Greater Possibilities!

Typical Memory Usage (Rough Ideas)

Data type	Memory per value	Example
int64	8 bytes	123
float64	8 bytes	1.23
bool	1 byte	True
object	50-200 bytes	"hello"
category	1-8 bytes	'A'
Int64 (nullable)	9 bytes	123 / <NA>

Different dtypes = Huge difference in memory usage!

Checking Memory Usage

- DataFrame.memory_usage()
- Series.memory_usage()

Example:

```
df.memory_usage(deep=True)
```

Index	Object
RangeIndex	122
name	580000
age	80000
city	4000000

Find where memory goes!

1 Object dtype

- Stores Python objects (strings, lists, custom objects).
- Very flexible but memory heavy.

Example:

```
df['name'].dtype  # object
```

Values	Memory (approx)
'A'	50-100 bytes
"New York"	100-200 bytes

Avoid object dtype for large data!

2 Numeric dtypes

- int8, int16, int32, int64
- float16, float32, float64
- Use the smallest type possible.

Example:

```
pd.Series([1,2,3], dtype='int8')
```

Type	Range	Memory
int8	-128 to 127	1 byte
int16	-32K to 32K	2 bytes
int32	±2B	4 bytes
int64	very large	8 bytes

3 Nullable dtypes

- Handle missing values (NA) with dedicated dtypes.
- Types: Int64, Float64, Boolean, String, etc.

Example:

```
pd.Series([1, None, 3],
 dtype='Int64')
```

✓ Better than object for missing values!

4 Categorical dtype

- Best for low-cardinality string columns.
- Store categories as codes (integer representation).

Example:

```
df['city'] = df['city'].astype('category')
```

Unique Values	Memory (approx)
100,000 (object)	~ 8 MB
100,000 (category)	~ 0.8 MB

Up to 10x less memory!

5 Downcasting

- Convert to smaller dtypes automatically.
- Use pd.to_numeric(..., downcast='...')

Example:

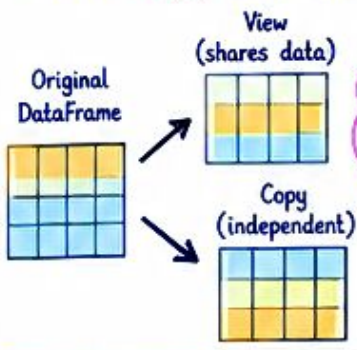
```
df['age'] = pd.to_numeric(df['age'],
 downcast='integer')
```

Before	int64 (8 bytes)
After	int16 (2 bytes)

Same Data Less Memory!

6 Copy vs View

- Slicing often returns a view.
- Modifying a view can change the original data.
- Use .copy() to be safe.



Use .copy() to avoid unexpected changes!

7 Memory Optimization Tips

- ✓ Use appropriate dtypes
- ✓ Convert object to category
- ✓ Downcast numeric columns
- ✓ Drop unnecessary columns
- ✓ Read only required columns
- ✓ Use efficient file formats (Parquet)
- ✓ Process data in chunks
- ✓ Avoid unnecessary copies
- ✓ Use in-place operations (where safe)

Small Changes = Big Impact!

8 10 GB Dataset → Manageable Memory Footprint

Step-by-step strategy:

1. Read only required columns

```
pd.read_csv('data.csv', usecols=[...])
```

2. Specify dtypes while reading

```
dtype={'id': 'int32', 'city': 'category', ...}
```

3. Use chunk processing if needed

```
for chunk in pd.read_csv('data.csv',
chunksize=10_00_000, usecols=..., dtype=...)
```

4. Optimize dtypes & categories
   - Downcast numeric columns
   - Convert low-cardinality strings to category
5. Save in Parquet for future use

```
df.to_parquet('optimized_data.parquet')
```

Before vs After

Original CSV (10 GB) → Optimized Memory ~ 1-2 GB (5-10x reduction!)

Same Insights, Much Lower Memory!

Real-World Example (Simplified)

Step	Dataset Size	Peak Memory	Notes
1. Original CSV	10 GB	12 GB	Many object columns
2. Read required columns + dtypes	10 GB	3.5 GB	Reduced columns
3. Downcast numeric + category	10 GB	1.8 GB	Optimized dtypes
4. Save as Parquet	1.8 GB	1.8 GB	Compressed on disk
5. Load from Parquet	1.8 GB	1.8 GB	Fast & efficient

10 GB → 1.8 GB = Same Analysis, Greater Possibilities!

Useful Functions

- df.info(memory_usage='deep')
- df.memory_usage(deep=True)
- df.select_dtypes(...)
- pd.to_numeric(..., downcast=...)
- df.astype('category')
- df.copy()
- pd.read_csv(..., usecols=..., dtype=..., chunksize=...)

Simple Functions, Powerful Results!

Key Takeaways

- ✓ Understand your data types.
- ✓ Avoid object dtype for large data.
- ✓ Use categorical dtype for low-cardinality strings.
- ✓ Downcast numeric columns.
- ✓ Read only what you need.
- ✓ Process large files in chunks.
- ✓ Use Parquet for big data.
- ✓ Monitor memory usage regularly.
- ✓ Good memory management enables bigger projects and better career opportunities!

Better Data Brighter Careers!

Golden Rules

- ✓ Use the smallest appropriate dtype.
- ✓ Avoid object dtype whenever possible.
- ✓ Check memory usage early and often.
- ✓ Parquet + proper dtypes + chunking = scalable analytics.

Optimize Memory Create Opportunities Keep Learning!



Under the Hood Today, Power Tomorrow!

Pandas Modern Architecture

Same Pandas ♥ Bigger Capabilities ♥ Real-World Impact

Not Just A Library... A Well-Engineered Ecosystem !

Why Understand Internals ?

- Explains Pandas behavior (sometimes surprising).
- Helps you write efficient code.
- Makes debugging easier.
- Prepares you for advanced use cases (big data, Arrow, custom dtypes).

Deeper Knowledge = Greater Possibilities!

The Big Picture

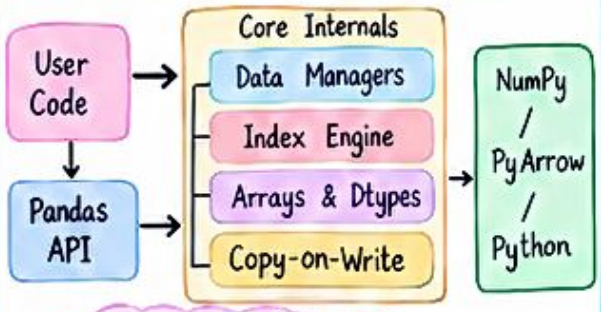
- Pandas = High-level API
- Built on powerful internal components
- Evolved from NumPy-only to a more flexible, Arrow-friendly, extension-based architecture.

Evolution of Pandas Internals

< 0.8	Simple, NumPy-based
0.8 - 1.x	BlockManager (default)
1.x	ExtensionArray, nullable dtypes
2.0+	ArrayManager (optional), Copy-on-Write, PyArrow integration, better dtypes

Same Goal: Fast, Flexible, User-Friendly Data Analysis!

High Level Architecture



A well-designed layered architecture!

1 Data Managers

- Responsible for storing data in DataFrame.
- Two managers:

BlockManager (default, legacy)

- Stores columns with same dtype in a single block.
- Memory efficient for wide DataFrames.

ArrayManager (experimental!)

- Stores each column as a separate array.
- Simpler, better for some use cases.

You don't need to choose - but know they exist!

BlockManager (column blocks)

Block 1 (int64)		Block 2 (float64)	Block 3 (object)
A	B	C	D
1	2	0.1	x
3	4	0.2	y
5	6	0.3	z

Columns with same dtype stored together

ArrayManager (one array per column)

Each column is a separate array

2 Extension Arrays

- A new interface for custom dtypes beyond NumPy.
- Enables features like nullable dtypes, Arrow arrays, categorical, string, etc.

More dtypes, greater flexibility!

3 Nullable dtypes

- Solve the missing value problem for non-float types.
- Use pd.NA (instead of np.nan)
- Keep the original dtype.

Type	Old (NumPy)	New (Nullable)
int	not possible	Int64
bool	not possible	Boolean
string	object + None	StringDtype
float	float64 + nan	Float64

Example:

```
pd.Series([1, pd.NA, 3], dtype="Int64")
```

Missing values without upcasting to float!

4 PyArrow Integration

- PyArrow provides efficient columnar memory format.
- Used in Pandas for:
  - ✓ ArrowExtensionArray
  - ✓ Parquet/Feather I/O
  - ✓ Better string and nested types
  - ✓ Zero-copy data exchange (with other tools like Spark)

Faster I/O, Lower Memory, Bigger Possibilities!

5 Copy-on-Write (CoW)

- Introduced in Pandas 2.0 (opt-in, going to be default).
- Avoids unnecessary copies.
- Data is only copied when you actually modify it.

Traditional (eager copy)	Copy-on-Write
df2 = df	df2 = df
↓	↓
Copy immediately	Share memory until modified

Same result, less memory, better performance!

6 DataFrame Internals

- A DataFrame is mostly:
  - Data manager (blocks/arrays)
  - Index (rows)
  - Columns (Index object)
  - Dtype information

Simple outside, Powerful inside!

7 Index Internals

- Index provides fast, immutable labels for rows/columns.
- Built on specialized engine (Hash/Sorted).
- Supports different types: RangeIndex, Int64Index, DatetimeIndex, CategoricalIndex, MultiIndex, ...

Immutable index = reliable & fast alignment!

8 Block/Array-Oriented Storage

- Pandas uses columnar, block-oriented storage for efficiency.
- Similar to database columnar storage and Apache Arrow.

Row-oriented (Traditional DB)

R1	[a1 b1 c1]
R2	[a2 b2 c2]
R3	[a3 b3 c3]

- ✓ Good for OLTP
- ✓ Row-wise access
- ✓ Slower for analytics

Columnar (Pandas/Arrow)

Col A	Col B	Col C
[a1	[b1	[c1
[a2	[b2	[c2
[a3	[b3	[c3

- ✓ Good for analytics
- ✓ Vectorized operations
- ✓ Better compression
- ✓ Lower memory

Real-World Analogy

- Books = Columns
- Shelf = Block/Array
- Library Catalog = Index
- Reading Without Copying = Copy-on-Write

Key Takeaways

- ✓ Data managers decide how data is stored.
- ✓ Extension arrays enable new dtypes.
- ✓ Nullable dtypes solve missing value issues.
- ✓ PyArrow brings modern, columnar capabilities.
- ✓ Copy-on-Write reduces unnecessary copies.
- ✓ DataFrame and Index have optimized internal structures.
- ✓ Columnar storage + Arrow = future-ready Pandas.

Understand Internals Use Pandas Better!

Remember

- ✓ You don't need to memorize internals.
- ✓ Just understand the concepts and why Pandas behaves the way it does.
- ✓ This knowledge helps you write faster, smarter and more scalable code, especially for large datasets.

Better Understanding, Greater Opportunities, A Brighter Data Career!

♥ Deeper Knowledge → Better Code → Larger Datasets → Bigger Impact → A Better Tomorrow! ♥

Keep Learning Keep Growing!



Two Libraries Stronger Together!

NumPy + Pandas Together

From Raw Data to Real-World Impact

Right Tool Right Problem Bigger Possibilities!

What is NumPy?

- Fast numerical computing library
- N-dimensional arrays
- Linear algebra, statistics, random numbers, FFT, etc.
- Foundation for scientific computing



NumPy = Speed for Numbers!

What is Pandas?

- Powerful data analysis library
- Works with tabular data (DataFrame, Series)
- Data cleaning, joins, grouping, time series, real-world data analysis



Pandas = Power for Data!

Together They Are Powerful

- ✓ Pandas uses NumPy under the hood.
- ✓ NumPy handles heavy numerical computations.
- ✓ Pandas provides high-level tools for real-world data.
- ✓ Best of both worlds!

NumPy + Pandas = Data Science Superpower!

When to Use What?

Use NumPy When

- Numerical computation
- Arrays & matrices
- Linear algebra
- Mathematical operations
- Random number generation
- High performance needed

Think: Numbers, Speed, Math!

Use Pandas When

- Tabular / structured data
- Data cleaning
- Joins / Merges
- Grouping & aggregation
- Time series analysis
- Data analysis & reporting

Think: Data, Insights, Real World!

End-to-End Workflow

1. Raw CSV (Real-world data)
2. Pandas DataFrame (Load & Explore)
3. Cleaning (Handle missing, types)
4. NumPy Numerical Operations (Fast computations)
5. Pandas Transformation (Add new features)
6. Aggregation (Summarize insights)
7. Visualization (See the story)
8. Export (Save results)

Same Workflow, Bigger Impact!

Complete Example: Sales Analysis Pipeline

1 Raw CSV → Pandas DataFrame

```
import pandas as pd
df = pd.read_csv('sales.csv')
df.head()
```

date	product	sales	cost
2024-01-01	A	100	60
2024-01-02	B	150	90
2024-01-03	A	200	120

Real data loaded!

2 Cleaning

- Handle missing values
- Convert dtypes
- Remove duplicates

```
df = df.dropna()
df['date'] = pd.to_datetime(df['date'])
```

Clean Data = Better Insights!

3 NumPy Numerical Operations

- Use NumPy for fast calculations

```
import numpy as np
df['profit'] = df['sales'] - df['cost']
df['profit_margin'] = np.round(
  df['profit'] / df['sales'], 2)
```

NumPy for Speed!

4 Pandas Transformation

- Feature engineering
- Create new columns

```
df['month'] = df['date'].dt.to_period('M')
df['sales_log'] = np.log1p(df['sales'])
```

New Features, More Value!

5 Aggregation

- Group, summarize, get insights

```
monthly = df.groupby('month').agg(
  total_sales=('sales', 'sum'),
  total_profit=('profit', 'sum'),
  avg_margin=('profit_margin', 'mean')
).reset_index()
```

From Data to Insights!

6 Visualization

- Use Pandas + Matplotlib/Seaborn

```
import matplotlib.pyplot as plt
monthly.plot(x='month', y='total_sales',
      kind='line', marker='o', title='Monthly Sales')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.grid(True)
plt.show()
```



7 Export

- Save results to CSV/Excel/Parquet

```
monthly.to_csv('monthly_summary.csv',
        index=False)
# or
monthly.to_parquet('monthly_summary.parquet')
```

CSV · Excel · Parquet

Before → After (Value Created)

Raw Data

date	product	sales	cost
01-01	A	100	60
01-02	B	150	90
01-03	A	200	120

Just data ...

Aggregated Insight

month	total_sales	total_profit	avg_margin
2024-01	450	180	0.40
2024-02	620	260	0.42
2024-03	800	360	0.45

... Now actionable insights!

Common Real-World Use Cases

- ✓ Sales & revenue analysis
- ✓ Customer segmentation
- ✓ Financial risk analysis
- ✓ Sensor & IoT data analysis
- ✓ Time series forecasting
- ✓ Scientific data analysis
- ✓ Machine learning preprocessing

Limitless Possibilities!

Key Takeaways

- ✓ NumPy gives you speed for numerical work.
- ✓ Pandas gives you power for real-world data.
- ✓ They work seamlessly together.
- ✓ Follow a structured workflow from raw data to exported insights.
- ✓ Use the right tool for the right problem.
- ✓ This combination is essential for data scientists, analysts and ML engineers.

Remember

- ✓ Don't just learn libraries, solve real problems.
- ✓ Combine NumPy + Pandas for bigger impact.
- ✓ Clean Data → Better Models
- ✓ Better Models → Better Decisions
- ✓ Better Decisions → A Brighter Tomorrow!

Good Data Good Analysis Greater Impact!

NumPy + Pandas = Learn Today Use Tomorrow = Build a Better World

Keep Learning Keep Growing!

Advanced Data Engineering with Pandas

Bigger Data Bigger Impact!

From Data Processing → Data Pipelines → Real-World Impact

Build Scalable Data Solutions Today!

Why Advanced Data Engineering?

- Real-world data is big, messy and comes from multiple sources.
- You need robust, scalable, reproducible pipelines.
- Pandas + modern tools help you build production-ready solutions.

Good Pipelines = Reliable Insights!

The Big Picture

Sources (CSV, DB, API) → Process (Pandas + Tools) → Store (Parquet/Arrow) → Use (Analytics/ML)

- ✓ Handle large data efficiently
- ✓ Ensure data quality
- ✓ Build maintainable ETL pipelines
- ✓ Scale with the right tools

Key File Format Ecosystem

Format	Best For	Key Features
CSV	Simple data	Human readable
Parquet	Big data	Columnar, compressed
Arrow	In-memory	Cross-language
Feather	Fast I/O	Lightweight Arrow

Parquet + Arrow = Modern Data Lake Foundation!

Core Principles

- ✓ Process in chunks
- ✓ Use columnar formats
- ✓ Validate data early
- ✓ Minimize memory usage
- ✓ Automate pipelines
- ✓ Use the right tool for scale
- ✓ Ensure data quality & reproducibility

Same Data, Bigger Possibilities!

1 Large CSV Processing

- Use proper options to handle large files efficiently.
- Specify dtypes, usecols, chunksize.

Example:

```
pd.read_csv('big.csv',
  usecols=['id', 'name', 'value'],
  dtype={'id':'int32'},
  low_memory=False)
```

Read only what you need!

Millions of Rows? No Problem!

CSV

2 Chunked Reading

- Read file in small chunks.
- Process each chunk and combine results.

Example:

```
chunks = pd.read_csv('big.csv',
chunksize=1_000_000)

total = 0
for chunk in chunks:
  total += chunk['value'].sum()
```

Save Memory Process Faster!

3 Parquet

- Columnar, compressed format.
- Faster read/write, smaller size.
- Best for analytics & data lakes.

Example:

```
df.to_parquet('data.parquet',
compression='snappy')
df = pd.read_parquet('data.parquet')
```

- ✓ Smaller size
- ✓ Faster I/O
- ✓ Preserves dtypes
- ✓ Industry standard

4 Arrow & PyArrow

- Apache Arrow = in-memory columnar format.
- PyArrow provides Python bindings.
- Used by Pandas, Polars, Spark, DuckDB.

Example:

```
import pyarrow as pa
import pyarrow.parquet as pq
pq.write_table(pa.Table.from_pandas(df),
 'data.parquet')
```

APACHE ARROW

Faster Data Everywhere!

5 Partitioned Datasets

- Store data in partitions (e.g., by date, region).
- Efficient for large datasets and selective queries.

Example (Hive style):

```
df.to_parquet(
  'data/',
  partition_cols=['year', 'month'],
)
```

data/
year=2024/
month=01/
month=02/
...

Organize for Scale!

6 Data Validation

- Use libraries like:
- pandera / great_expectations / custom checks

Example (pandera):

```
import pandera as pa
schema = pa.DataFrameSchema({
  'id': pa.Column(int),
  'value': pa.Column(float)
})
schema.validate(df)
```

Valid Data = Trusted Insights!

7 ETL Pipelines

- Extract → Transform → Load
- Automate with functions/classes.
- Can use workflow tools (Airflow, Prefect) for scheduling.

Extract (CSV/DB/API) → Transform (Clean/Enrich) → Load (DB/Parquet)

Small Steps, Big Impact!

8 Memory-Efficient Processing

- Use appropriate dtypes
- Downcast numeric columns
- Use categorical for low-cardinality
- Process in chunks
- Avoid unnecessary copies
- Use vectorized operations
- Leverage Parquet/Arrow

Less Memory More Data More Insights!

9 Data Quality Checks

- Check missing values
- Check duplicates
- Validate ranges & constraints
- Check consistency across columns
- Use summary reports

Common Checks

```
df.isnull().sum()
df.duplicated().sum()
df.describe()
df.nunique()
```

Clean Data Better Decisions!

When Data is Too Large for Pandas → Powerful Alternatives

1 Polars

- Blazing fast DataFrame library written in Rust.
- Low memory usage, multi-threaded.
- Similar API to Pandas (easy to learn).

Great for single machine large datasets!

```
import polars as pl
df = pl.read_parquet('data.parquet')
```

Polars ✓ Fast ✓ Modern ✓ Memory Efficient

2 Dask

- Parallel computing library.
- Scales Pandas-like API to multi-core & distributed clusters.
- Good for bigger-than-memory datasets.

Scale Pandas with Dask!

```
import dask.dataframe as dd
df = dd.read_csv('*.csv')
```

Dask ✓ Parallel ✓ Scalable ✓ Flexible

3 DuckDB

- In-process analytical database.
- Query large files (CSV/Parquet) with SQL.
- Great for analytics without complex setup.

SQL on your local files!

```
import duckdb
duckdb.sql(" SELECT * FROM
    "data.parquet" LIMIT 5").df()
```

DuckDB ✓ Fast SQL ✓ No Server ✓ Works with Pandas

4 Spark

- Distributed data processing engine.
- Handles massive datasets across clusters.
- Popular in industry for big data and ML.

For Truly Massive Scale!

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()
```

Spark ✓ Distribute Compute ✓ Big Data Ecosystem

Comparison at a Glance

Tool	Best For	Scale	Ease of Use
Pandas	Small-Medium	1 machine	Easiest
Polars	Medium-Large	1 machine	Easy
DuckDB	Large (SQL)	1 machine	Easy
Dask	Larger-than-memory	Multi-core/Cluster	Moderate
Spark	Very Large	Cluster	Complex

Key Takeaways

- ✓ Use Pandas for small-medium data.
- ✓ Use Parquet + Arrow for efficient storage.
- ✓ Process large CSVs in chunks.
- ✓ Validate data and ensure quality.
- ✓ Build robust ETL pipelines.
- ✓ When data grows, choose Polars, Dask, DuckDB or Spark.
- ✓ Right tool for the right scale = success!

Don't just handle data, build data products!

Golden Rules

- ✓ Read only what you need.
- ✓ Use columnar formats (Parquet/Arrow).
- ✓ Validate data early.
- ✓ Automate and monitor pipelines.
- ✓ Scale with modern tools.

Better Data Brighter Opportunities A Greater Tomorrow!

Process Data ♥ Build Pipelines ♥ Scale Solutions ♥ Create Impact ♥ Keep Learning